

HD²RNet: hybrid drift-enabled distributed recurrent network for intrusion detection in edge computing

Abdul Junaid Mohammed, M. Sadhasivam, Rahmat Ali & Kuna Sneha

To cite this article: Abdul Junaid Mohammed, M. Sadhasivam, Rahmat Ali & Kuna Sneha (23 Mar 2026): HD²RNet: hybrid drift-enabled distributed recurrent network for intrusion detection in edge computing, Cyber-Physical Systems, DOI: [10.1080/23335777.2026.2646840](https://doi.org/10.1080/23335777.2026.2646840)

To link to this article: <https://doi.org/10.1080/23335777.2026.2646840>



Published online: 23 Mar 2026.



Submit your article to this journal [↗](#)



Article views: 24



View related articles [↗](#)



View Crossmark data [↗](#)



HD²RNet: hybrid drift-enabled distributed recurrent network for intrusion detection in edge computing

Abdul Junaid Mohammed^a, M. Sadhasivam^b, Rahmat Ali^c and Kuna Sneha^d

^aDepartment of Information Technology, University of the Cumberlands, Williamsburg, KY, USA;

^bDepartment of Artificial Intelligence and Data Science, Panimalar Engineering College, Chennai, India;

^cTechnology Lead Department of Information Technology, University of the Cumberlands, Williamsburg, KY, USA;

^dDepartment of Computer Science and Engineering-Data Science, Sphoorthy Engineering College, Hyderabad, Telangana, India

ABSTRACT

Edge computing provides efficient processing however, the distributed and heterogeneous nature of IoT-edge systems brings forth challenges in terms of security, interoperability, and resource management. Therefore, this research proposes, an improved intrusion detection system termed the Hybrid Drift enabled Distributed weight-tuned Recurrent Network (HD²RNet) to detect and classify intrusions accurately to ensure secure operation and protect sensitive data in edge environments. During performance evaluation, the HD²RNet obtained a 97.68% accuracy, a 97.26% recall, 98.16% precision, and 97.71% F1 score on the TON-IoT dataset respectively.

ARTICLE HISTORY

Received 7 August 2025

Accepted 28 February 2026

KEYWORDS

Edge computing; intrusion detection; internet of things; drift mechanism; deep learning

1. Introduction

The development of the Internet of Things (IoT) has increased drastically over the past few years, leading to the improvement of wireless communication and the processing of information [1]. This growth has spurred an edge computing growth across the globe, with estimates showing close to 6.5 billion devices by the year 2030 [2]. However, the introduction of these technologies also gives rise to a lot of security issues, encompassing threats like malware, exploits, Distributed Denial-of-Service (DDoS) attacks, and even backdoors, that disrupt the security of smart environments and connected devices [3,4]. Due to the essential role that the current computing systems play, it is necessary to utilise advanced security systems and intrusion detection policies that will help identify and prevent malicious activities [5,6]. Intrusion Detection Systems (IDS) are configured to detect possible threats or malicious operations so that immediate actions can be taken. The topic of security should be rather advanced, with the possibility to detect and avoid harmful activity such as Man-in-the-Middle (MitM) attacks, Denial of Service (DDoS) [7], privacy disclosure, and privilege escalations [8]. An IDS tracks network traffic and device activities, which serves as a critical layer of defence. The ongoing development of new forms of attacks requires constant modification and screening of IDS in constantly

changing network structures to guarantee a high level of network security and stable functioning of the IoT ecosystems [4].

The traditional types of cyberattacks, including eavesdropping, unauthorised data interception, and intermediary-based intrusion attacks, are relatively dangerous to network security. The first level of securing against these threats in any architecture is the use of IDS, which can identify and mitigate the threats on any level of three-tier edge computing. IDS inspects the traffic that passes through a network, tracks data flows, and notices the pattern of the nodes, cloud servers, and IoT devices as a way to pre-empt an attack [2]. In contrast to traditional firewalls, the purpose of IDS is to recognise a wide range of malware as early as possible. The emergence of better IDSs is of utmost importance owing to the increased volume of threats [6]. The majority of IDSs are categorised into two classes, including Signature-based and anomaly-based methods [9]. The signature-based IDS (SIDS) recognises known threats by matching data flow, whereas anomaly-based IDS (AIDS) detects unknown or abnormal activity through traffic patterns [10]. Nevertheless, most IDS solutions rely on machine learning algorithms to identify malicious nodes, often neglecting data integrity, leading to a greater risk [11]. Besides, central SDN controllers, which are typically encountered in IDS systems, add single points of failure and complexity. AIDS can identify unknown attacks, despite tending to result in a significant number of incorrect positive detections. Traditional IDS systems struggle to effectively identify attacks, leaving smart devices exposed to privacy breaches and location tracking by malicious entities [12].

Edge paradigms, including the mobile edge computing (MEC) and fog computing (FC), are emerged to address the IoT implementation challenges. Specifically, the nodes that implement computing tasks are referred to as MEC hosts or fog nodes in MEC and FC hosts in MEC, which provide lower-latency services. MEC hosts are installed through mobile service providers, while FC involves an edge server or device with computing and communication power [7]. IDS for fog and edge computing environments identifies the intruders using two methods: anomaly-based detection and signature-based detection. Anomaly-based detection considers the normal behaviour of the network traffic and then observes the behaviour of incoming traffic and classifies it as either normal or abnormal. In contrast, signature-based methods compare the incoming traffic with pre-established rules to detect the attack patterns in the network [13]. Being a decentralised solution, edge computing accomplishes data processing on the edge of the network and then transmits data to the cloud, thereby increasing the responsiveness and performance of the IoT devices [5]. Nevertheless, the process of securing Mobile Edge Computing (MEC) settings becomes very challenging. The distributed nature of MEC expands the attack surface, requiring robust security measures to protect communication channels and data exchanges [14]. The drawbacks of the Traditional IDSs include a slow response time, speed limit during transfer, and inability to adjust to large and scattered networks [15]. Deep learning models are increasingly preferred models in implementing IDSs due to the ability to bypass manual engineering of features and require lower input specifics of a domain, and therefore, the models are applicable in next-generation IoT networks [16]. However, IoT endpoint devices are also prone to attack, which makes advanced and resilient intrusion detection systems an essential part of information privacy and security at the endpoint [17].

To mitigate these issues, this research designs a proposed HD²RNet framework for intrusion detection in IoT-edge ecosystems to strengthen protection within edge-based IoT scenarios. The main benefit of this research lies in its concept of edge computing, which is characterised by a minimal dependence on a remote cloud infrastructure by minimising latency by processing data near the source. By aggregating network behaviour patterns from diverse IoT edge devices, the system leverages a broader context for efficient learning and detection of network patterns. The incorporation of a distributed architecture allows high scalability and eliminates any single-point failure by processing the input features of networks in parallel, and the use of adaptive mechanisms enhances the system's capability to respond to dynamic and evolving threat landscapes typical of IoT systems. In addition, the framework enhances efficiency in learning through appropriate filtering and selection of relevant features during preprocessing. Overall, the framework enhances the detection by mitigating false alarms and supports proactive threat mitigation at the edge layer, thereby enhancing the security of decentralised IoT environments.

Hybrid Drift enabled Distributed weight-tuned Recurrent Neural Network: The HD²RNet incorporates a distributed network of RNN layers to extract time-based intrusion patterns sequentially to enable localised learning of traffic patterns and the parallel processing capability. Moreover, the system employs a drift detection mechanism, which allows it to adapt dynamically to various data drifts and intrusive behaviour. This enables the framework to adapt to new intrusion behaviours over time. Overall, the hybrid architecture integrates a combination of temporal learning, adaptive response to the drift and distributed processing, significantly enhancing the protection of the IoT-edge infrastructures.

The paper is organised in the following manner. [Section 2](#) presents an overview of the related literature, problem formulation, and issues. [Section 3](#) depicts the systematic approach used by the proposed IDS. [Section 4](#) explains the mechanisms adopted by the proposed HD²RNet framework. The discussion on results and comparisons is provided in [Section 5](#). [Section 6](#) presents the conclusions along with directions for future research.

2. Literature survey

The prior literature regarding intrusion detection using edge computing architecture is demonstrated below. Mahadevappa et al. [2] conceptualised the use of ML approaches like Linear Discriminant Analysis (LDA) and Logistic Regression (LR) to efficiently identify the intrusion along with edge computing. Additionally, for the isolation of suspicious devices, a different mechanism is employed to classify attacks. While testing the model, it significantly surpassed other models by reporting a 96.6% accuracy and 95.7% precision; despite this, the model faced scalability and enhanced costs for computation. Bhavsar et al. [1] designed an IDS based on Federated Learning, termed the (FL-IDS), to improve vehicle network security using IoT edge devices. The model utilised the Pearson coefficient and the CNN to understand the attacks on transportation. The model, when tested on the Car-Hacking database, showcased a 99.7% accuracy. However, the framework obscured overhead issues and decreased convergence.

Alsubhi [17]. suggested a Secured Edge Computing-based IDS System (SEC-IDS) especially for MEC platforms. The approach included anomaly and signature-based tasks to

enhance the detection of intrusions. The approach provides minimal latency with profound capability of detection accuracy. However, the security is lacking with prolonged computation costs, which is common in edge computing environments. The IDS system developed by Javeed et al. [18] incorporated a Truncated Back Propagation through time (BPTT) in relation to Bi-directional Gated Recurrent Unit (Bi-GRU) and LSTM to detect intrusions in an agricultural environment. This approach uses timestamps of the network to extract the sequence length and determine the attack and classification using recurrent models. The model, when measured using the CIC-IDS2018, delivered a class detection rate of 99.7%. However, the limited resources lead to overfitting issues.

Tariq et al. [19] incorporated edge computing-enabled Support Vector Machine along with the FL for identifying the intrusions, especially affecting the smart grid infrastructure. The FL provides an improved standard for training edge devices while introducing collaborative learning. While validated on the CICIDS database, an accuracy of 94% is attained, reflecting its efficiency. The framework lacked the adaptability to edge devices and leading to high false alarm rates. Hoang et al. [3] applied the Distributed Machine Learning strategies (DML) along with edge computing to detect attacks imposed by the diverse networks. The algorithm leads to high accuracy, minimal connectivity bottleneck, and data privacy. Nevertheless, finding the effective trade-off between the number of resources and maintaining accuracy remains an open issue. The outcomes, when evaluated on the UNSW-NB15, showcased a detection rate of 95.06% in addition to various other attacks.

Kipongo et al. [11] presented a Transfer Learning-based Deep Q-Network (TLDQN) which comprises strategies of the cube algorithm, routing based on reinforcement learning, and the modified honeycomb for network division to detect malicious nodes on the network. Moreover, the bidirectional adversarial networks and the Driver-Trainer-based Optimisation (DTO) improved the solution performance. However, the model incurred overhead while transferring the data through edge servers. Aldaej et al. [5] improvised the ensemble approach consisting of Random Forest (RF), Deep Neural Network (DNN), and E-Tree to measure the unexpected events of the networks. Meanwhile, the four network-traffic enabled datasets are employed for classification, where on the NSL-KDD, the framework produced a precision and a recall of 94.4% and 94.6%, respectively. Despite these, the framework still suffers from computational overhead and overfitting issues.

Alzubi et al. [20] presented the IDSs exploiting the Artificial Neural Network (ANN) and Salp swarm algorithm (SSA) that improved the intrusion detection in the IoT environment. In the SSA-MLP method, the SSA algorithm selected the optimal parameters and identified the most informative features that highly contributed to boosting the detection performance. However, the system required more training time and faces scalability issue with evolving network traffic.

Alzubi et al. [13] implemented the Effective Seeker Optimisation coupled with Machine Learning-Enabled IDS in the fog and edge computing environments. Specifically, the ESOML-IDS model utilised the ESO algorithm for feature selection and selected the optimal subset of features to detect the presence of intrusions in the network traffic. Moreover, the ESOML-IDS was highly applicable for extracting the diverse benefits with a minimal loss in accuracy for intrusion detection.

Alzubi et al. [21] introduced the blended DL-based intrusion detection framework that exploited the strengths and capabilities of different DL architectures. Further, the blended

DL combined CNN and LSTM architectures to identify the latest intruders precisely and elevated the security level of healthcare data.

In order to address the drawbacks in the existing methods, the proposed HD²RNet combining the distributed learning mechanism, dropout layer, and drift detection, highly contributes to identifying the drifts in streaming network traffic. Specifically, the proposed architecture, coupling the distributed learning strategy, offers parallel processing and real-time analysis of large-scale traffic data to capture different patterns in the network traffic. In addition, the distributed learning strategy works collaboratively with the concept drift phenomenon to identify the dynamic variations in the network traffic, improving the overall detection capability. Consequently, the proposed network detects drifts quickly and with minimal memory usage, promoting the IDS to rapidly adapt to evolving network environments and identify unknown attacks.

2.1. Challenges

This section lists some of the issues inherited by the previous approaches in the domain of intrusion detection.

- Due to the incorporation of sophisticated architecture, the LDA-LR approach [2] faced issues, such as scalability and enlarged computational resources.
- The major issue imposed by FL-IDS [1] is the communication overhead and the instability in convergence rate, which subsequently leads to performance degradation.
- The SEC-IDS [17] enhanced the computational burden and, at the same time, it provided low latency and adaptability issues due to the constantly changing edge computing environments.
- The edge-envisioned framework BPTT [18] failed to maintain a reliable detection capability in an extreme environment with limited available edge resources and intermittent network access.
- The TLDQN [11] model possesses issues, including large resources and overfitting, thus necessitating thorough tests of feasibility before practical implementation.

Despite the promising efforts of the existing solutions, there still exist critical issues affecting the intrusion detection. More specifically, the existing methods exhibit certain challenges, including low training efficiency, overfitting issues, complex operation, and latency issue, and lack the sophisticated mechanism to capture the complex correlation between features, leading to high false-positive rates. However, the proposed HD²RNet addresses the above-mentioned challenges in the conventional methods with the application of the distributed learning mechanism, dropout layer and drift detection, enhancing the generalisation ability of the network and highly contributing to reducing the risk of overfitting. Specifically, the distributed learning strategy enables the proposed network to handle large datasets, improving the scalability of the model. Further, the proposed network reduces the computational burden through parallel processing, and improves the training efficiency. Besides, the application of drift detection facilitates the proposed network to capture the dynamic variations in the network traffic to analyse the attack

patterns and reduce the false-positive rates, making the model highly applicable for real-time intrusion detection.

2.2. Problem statement

As the adoption of IoT devices expands rapidly in different fields, there is a necessity to introduce efficient and secure mechanisms. Edge computing has become a potential paradigm with low latency for data processing, making the computation closer to the IoT devices. Edge computing allows real-time decision-making and reduces the burden on centralised cloud systems. Nevertheless, edge computing environments are highly prone to cyber intrusions because of their resource-constrained and heterogeneous nature. The protocols that IoT edge devices use for communication include different levels of the Open Systems Interconnection (OSI) model, such as data link, network, physical, session, transport, presentation, and application levels. The data that is created and processed by the edge devices typically pass through these layers, and it is grouped or passed on to the cloud so it can be further analysed. Among them, the network, transport, and application layers are more susceptible to some of the well-known attacks on the network, namely Denial of Service (DoS), spoofing, Cross-site scripting (XSS), reconnaissance, and many more. These attacks can compromise data integrity, availability, and confidentiality, thereby compromising the whole IoT infrastructure. Moreover, the existing centralised IDSs are not suitable for this dynamic environment due to the large latency, bandwidth limitations, and non-scalability. Thus, there is an urgent necessity for distributed and intelligent intrusion detection mechanisms, especially for edge computing-based IoT systems. This research proposes an edge computing-based intrusion detection approach that aggregates and analyses network behaviour patterns by constructing representative datasets like the TON-IoT dataset [22] and BoT-IoT dataset [23]. The collected network traffic data E can be denoted as,

$$E = \{E_1, \dots, E_u, \dots, E_v\} \quad (1)$$

where E_u implies the u^{th} edge device data and E_v denotes the total 'v' number of edge device data that contains multi-layered network traffic characteristics, which can be represented as,

$$E_u = \{e_{u,1}, e_{u,2}, \dots, e_{u,j}\} \quad (2)$$

where $e_{u,j}$ implies the u^{th} network data patterns with a total 'j' number of patterns that determine data transmission. These raw network traffic data undergo preprocessing, where irrelevant components unnecessary for analysis are removed and transformed into qualitative data for processing. The outcomes following this step can be expressed as 'R'. The following data undergoes model training, thus enabling efficient detection of evolving intrusion patterns. Moreover, the performance reduction can be managed by minimising the error value through employing categorical-cross entropy (γ_{loss}) as loss function.

$$y_{loss} = - \sum_{m=1}^M z_m^* \log(z_m) \quad (3)$$

Here, M denotes the whole count of classes, z_m and z_m^* portrays predicted and actual labels of the m^{th} class. The actual labels in the TON-IoT can be denoted as,

$$z_m^* = \begin{cases} 0; Benign \\ 1; Scanning \\ 2; DDOS \\ 3; XSS \\ 4; Password \\ 5; DOS \\ 6; Injection \end{cases} \quad (4)$$

Similarly, for the BoT-IoT dataset, the actual labels z_m^* can be represented as,

$$z_m^* = \begin{cases} 0; Normal \\ 1; Reconnaissance \\ 2; DDOS \\ 3; DOS \\ 4; Theft \end{cases} \quad (5)$$

3. System model of intrusion detection

In this research, an intrusion detection system tailored for edge computing environments is presented to enhance the security of IoT-based infrastructures. Figure 1 depicts the systematic approach of intrusion detection. The system at the early stage considers the deployment of several edge devices, like sensors, actuators, and smart appliances that constantly generate and transmit data. This information is forwarded to the edge layer, where the data is transmitted to the cloud environment, ensuring low-latency processing and minimal resources. However, owing to the distributed and limited computational resources of the edge layer, it becomes a major target for intruders who attempt to exploit communication protocols and device vulnerabilities. To resolve this, network data patterns generated at the edge layer, both normal and malicious, are continuously monitored and aggregated into a centralised database comprising the TON-IoT [22] and the BoT-IoT dataset [23]. The aggregated data undergoes preprocessing to eliminate noise, irrelevant data, and transform the data into enhanced quality. Then, the dataset is partitioned into training and testing subsets. The train set is utilised for training the IDS, and the testing set, once the model gets trained, is used to assess the model's effectiveness by detecting the presence of intrusions by classifying the specific type of intrusion in network traffic.

4. Proposed hybrid drift-enabled distributed weight-tuned recurrent neural network for intrusion detection in IoT-edge environments

The proliferation of IoT environments generates vast amounts of information, requiring rapid processing and analysis to support intelligent decision-making. Edge computing has evolved as a complementary architecture that moves processing, storage, and analysis

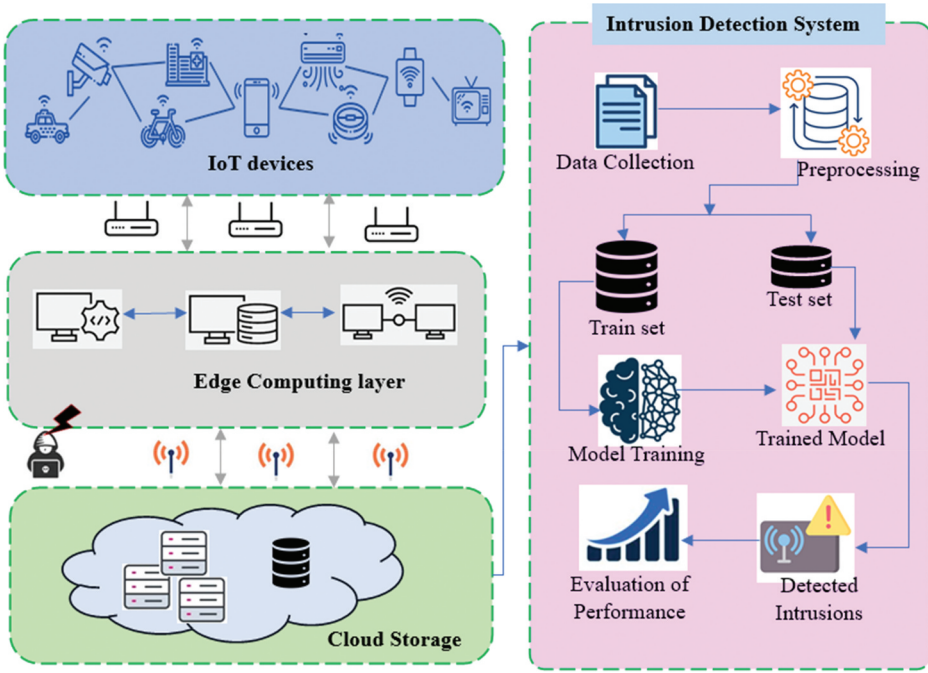


Figure 1. System model of the proposed IDS in IoT-Edge environment.

functions nearer to the data source at or near the IoT devices. Despite its benefits, the distributed and heterogeneous nature of IoT-edge systems brings forth challenges in terms of security, interoperability, and resource management. Therefore, this research is aimed at proposing an HD²RNet framework to detect intrusions in an edge-computing environment to strengthen the protection of IoT-based edge systems. Initially, the network data patterns from the diverse edge devices are aggregated into a database like the TON-IoT [22] and the BoT-IoT dataset [23] that contains network activities, replicating both normal and malicious behaviour by initiating the request to the cloud, which shares the edge server data like network parameters essential for detecting intrusions. This aggregated data is fed into preprocessing for the elimination of redundant and irrelevant features to increase the effectiveness of learning. The pre-processed data are then categorised into testing and training sets. The training data is input into the proposed HD²RNet model, which is a distributed network where multiple layers of RNN will be deployed to extract essential dependencies. Additionally, the HD²RNet utilises the Drift detection mechanism, which checks the statistical changes in the input patterns with the help of parameters based on the Beta distribution. This allows the model to adaptively recognise changes in network behaviour that may indicate evolving or previously unseen attacks. Once trained, the model detects the presence of intrusions by classifying the specific type of intrusion in network traffic. Finally, the model's effectiveness is evaluated through key performance measures, accuracy, sensitivity, specificity, and detection rate, to determine the performance in handling dynamic intrusion patterns of IoT-edge environments. Finally, the IDS shares responses such as detected intrusion or normal and is stored in the cloud, which is further shared with edge devices to be aware of malicious

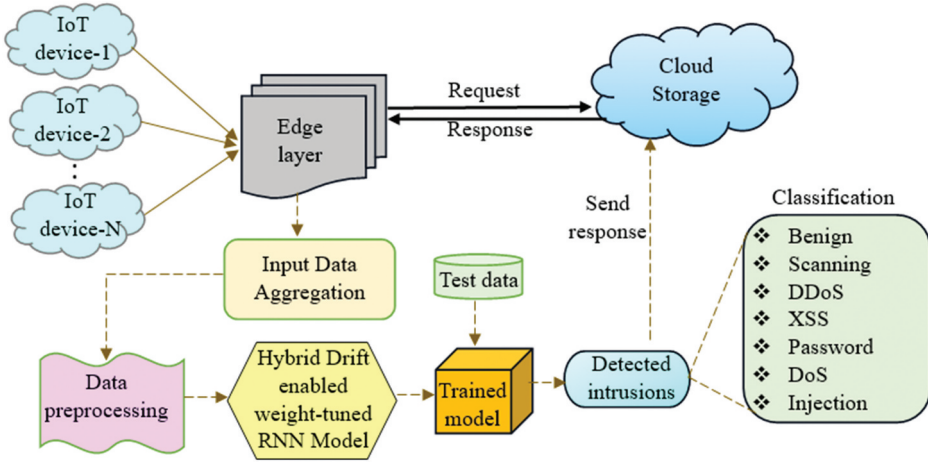


Figure 2. Overview of the proposed HD²RNet-based IDS.

attacks. This method reduces detection latency while improving the overall security of IoT-edge systems by enabling early identification of abnormal network behaviour. Figure 2 presents the schematic representation of the proposed framework.

4.1. Input network traffic data

In the initial step, the varied network patterns are collected through edge-based IoT systems through cloud storage upon request, and the obtained data are gathered from databases, like the TON-IoT [22] and BoT-IoT dataset [23], representing different characteristics of the network patterns essential for detecting intrusions. The representation of the datasets is expressed in Equations (1) and (2), and the input data E_u is passed further to pre-processing, which represents a size of $(N, 45)$ for the TON-IoT dataset and for the BOT-IoT dataset $(N, 20)$, correspondingly.

4.2. Preprocessing of network patterns

In the preprocessing, the aggregated traffic network data E_u is passed as input to eliminate irrelevant and redundant columns. Usually, a network traffic dataset typically encompasses multiple features, such as timestamps, IP addresses of both sources and destinations, protocol flags, packet length, service types, and many more. During the preprocessing, the irrelevant features were selected based on the columns containing high unique value having no effect on the output being removed from the input data. Especially, the presence of irrelevant columns not only impacts the learning process but often introduces noise and overfitting. Hence, the removal of such non-informative features enhances the computational effectiveness of the model since it lowers the dimensionality of the input data. Hence, the preprocessed data after removing unwanted columns can be denoted as R , having the dimensions of $(N, 41)$ for the TON-IoT dataset and $(N, 14)$ for the BOT-IoT dataset.

4.3. Proposed hybrid drift enabled distributed weight-tuned recurrent neural network for intrusion detection

Existing RNN-based IDS captured the complex attack sequences, resulting in higher accuracy, and reduced the dependence on manual feature engineering. However, the RNN model is highly susceptible to overfitting during training, causing significant challenges in identifying the new attack pattern in the network traffic [24]. In addition, the existing models required higher training time, complex operations, and were incompetent for generalising the newer attack patterns [25]. In contrast to LSTM and GRU architectures, the proposed method exploiting the RNN network provides a simple architecture, resulting in faster training times and minimising the computational costs. Specifically, the core of the RNN network lies in the recurrent computation of the hidden state, allowing the RNN architecture to model the temporal dependencies and trends in sequential data. Consequently, this research proposes the HD²RNet model coupling the RNN architecture with a distributed learning mechanism and drift detection module to address the drawbacks in the existing methods and significantly contribute to capturing the dynamic variations in network traffic. Further, the incorporation of dropout layer combats the overfitting issue, leading to improved generalisation performance. Additionally, the distributed learning strategy combined with the concept drift phenomenon effectively analyses the dynamic variations in the network traffic, contributing to enhancing the overall detection performance.

In the research, the HD²RNet framework used for the intrusion detection in the edge computing infrastructure incorporates a distributed architecture consisting of multiple RNN layers, dropout, and dense layers to capture the inherent features of the network parameters. Additionally, the concept of drift is implemented to tackle the model performance degradation, especially when the correlations between the input features and the target outcome change rapidly. Collectively, these strategies support increasing the detection accuracy, monitoring, and tracking of the data distributions in the framework. The architecture of the HD²RNet model initially allows the pre-processed input R of size $(N, 41)$ as input to the simple RNN layers, which is distributed into two parts, for parallel processing of network parameters to efficiently learn the dependencies between the data. The pre-processed input is reshaped into $(N, 41, 1)$ and is passed as input to the RNN layer of dimension $(N, 41, 10)$, which learn the dependencies among the diverse network patterns and predicts network behaviour based on past observations, making them ideal for detecting both known and emerging threats. The set of RNN layers with dimension $(N, 20)$ is applied on both ends of the distributed network. The concise overview of the RNN is elaborated in the next part.

4.3.1. Distributed recurrent neural network

The RNNs [26] are a form of DNN especially incorporated for handling sequential and time-related data dependencies, making them suitable for detecting diverse network patterns across timesteps. In IoT-based edge environments, network traffic flows where various events or packets are imagined as a sequence by the RNN across time, containing both normal and malicious activities. Since intrusions such as DoS attacks, port scanning,

or malware injections often unfold over time, detecting them requires models that can learn from past contextual information. Meanwhile, RNNs are far better than feed-forward networks in understanding patterns. RNNs [27] maintain an internal state memory through recurrent connections, which means learning in a loop to enable the model to understand temporal connections and dependencies of the patterns. This capability is recognised as crucial in recognising sophisticated intrusions that may be spread across multiple time durations or hidden within the normal sequences, thus affecting the edge-based systems of the IoT infrastructure. Therefore, integrating RNNs into intrusion detection frameworks, especially in edge computing, offers a robust solution for efficient, context-aware, and adaptive threat identification.

The reshaped outcome R determining a dimension of $(N, 41, 1)$ is fed into the RNN. At each time step p , the RNN receives an input vector R_p , which passes through an internal memory state known as the hidden state or context-state c_p that maintains data information of the previous timestep c_{p-1} in the sequences and updates the hidden state to produce the outcome e_p . This enables the model to remember previous inputs and capture time-dependent patterns in network traffic. The RNN computation can be expressed as,

$$c_p = \tanh(Q_{Rc}R_p + Q_{cc}c_{p-1} + h_c) \quad (6)$$

$$e_p = \sigma(Q_{Re}c_p + h_e) \quad (7)$$

where Q_{Rc} , Q_{cc} , and Q_{Re} represent weight matrices of the input-hidden, hidden-hidden, and the hidden-output, whereas h_c and h_e denotes the bias vector of the hidden and the output state. R_p indicates the pre-processed outcome at timestep p , $\tanh$ implies the tangent hyperbolic function, and σ denotes the sigmoid activation. This is particularly helpful in intrusion detection, where a single network parameter may not reflect malicious behaviour, but the sequences of parameters can reveal vulnerabilities. RNNs can learn these essential dependencies using the past behaviours captured by the hidden state. This memory state enables the network to retain prior information and adjust dynamically in response to new data, which makes it ideal for the detection of intrusions more accurately. Additionally, RNNs can be trained to classify the type of intrusion by analysing the context within a sequence, thereby enhancing the system's ability to accurately identify forgeries. The hidden state ' c_p ' of dimension $(N, 20)$ obtained after passing through two levels of RNN layers is forwarded to the next stage of the dropout layer.

4.3.2. Dropout layer

After passing through RNN layers, the dropout layer of dimension $(N, 20)$ is implemented at both sides of the distributed network. The dropout layers are introduced generally in the architecture to randomly terminate some of the active neurons by setting a certain range of activations to zero while training. It also acts as a form of regularisation technique to address the overfitting issue, where the model excels on trained data and performs poorly on unseen data. By random deactivation of certain activations, the dropout forces the network to learn more generalised features rather than memorising specific patterns, leading to increased generalisation.

4.3.3. Dense layer

The dense layers of dimensionality $(N, 64)$ and $(N, 128)$ is applied twice after the dropout layers to understand the intricate patterns and relationships within the network. It also enables interactions between all the input features of the model to improve learning by introducing non-linearity among the features. This is especially important in IDS, where detecting malicious activity often involves diverse variations across network traffic or system logs. The dense layers also expand or minimise the feature dimensions by adjusting the count of activations.

Then, the outcome from the distributed networks at both ends is concatenated into a single layer representing a dimension of $(N, 256)$, which is then applied to the dense layer to attain the classification outcome as a predicted probability outcome (p_m) of dimension, $(N, 7)$ where each class determines the type of intrusion. Furthermore, the HD²RNet framework reduces the prediction loss through the loss function, defined in Equation (3). During the testing phase, the model demonstrates strong performance during the training phase; however, when the newer test data $(\hat{R})$ is applied, if there exists any deviation in performance, the model may detect the deviations, which can be represented as drift. Therefore, to handle data drift, a beta distribution-based drift detection mechanism is utilised. The operation of the drift mechanism is detailed in Section 4.4. Figure 3 illustrates the architectural diagram of the HD²RNet framework. Finally, if the model detects the traffic patterns as an anomaly, then the proposed IDS model alarms the cloud storage and the subsequent edge system through a response message as 'warning' to stay aware of intruders who transmit malicious attacks by requesting data; otherwise, the response is provided as 'safer'.

$$Response = \begin{cases} "warning"; & \text{if } E_u \text{ considered anomaly} \\ "Safe"; & \text{if } E_u \text{ considered normal} \end{cases} \quad (8)$$

4.3.4. Drift mechanism based on beta distribution

Data drift [28] can be defined as the phenomenon where the performance of the model degrades over time due to alterations in the data distribution, while the dependencies among the input features and the target variable change rapidly. Therefore, detecting drifts is essential to maintain reliability, accurate performance, and stability of the model. Data drift detection is the process of recognising the changes in the data properties or behaviour over time and identifying the deviation from their expected state. Hence, the HD²RNet framework utilises a beta distribution-based sliding window technique for data drift detection.

Comparatively, the entropy-based method requires the nodes to have a very low packet loss rate to attain the high trust value. Despite the fact that the entropy-based method allows faster detection of attack victims, while increased number of lost packets causes the trust value to be reduced further. Nevertheless, the computational complexity associated with the entropy-based method is higher than that of the beta distribution [29]. Further, the Kullback-Leibler (KL) divergence methods fail to provide enhanced intrusion detection due to the presence of outliers and increase the computational overhead. As a result, the beta distribution is highly preferred in intrusion detection systems that reduces the computation overhead, offers flexibility, and is highly robust to outliers. Beta distribution is a continuous probability distribution that enables adaptability in

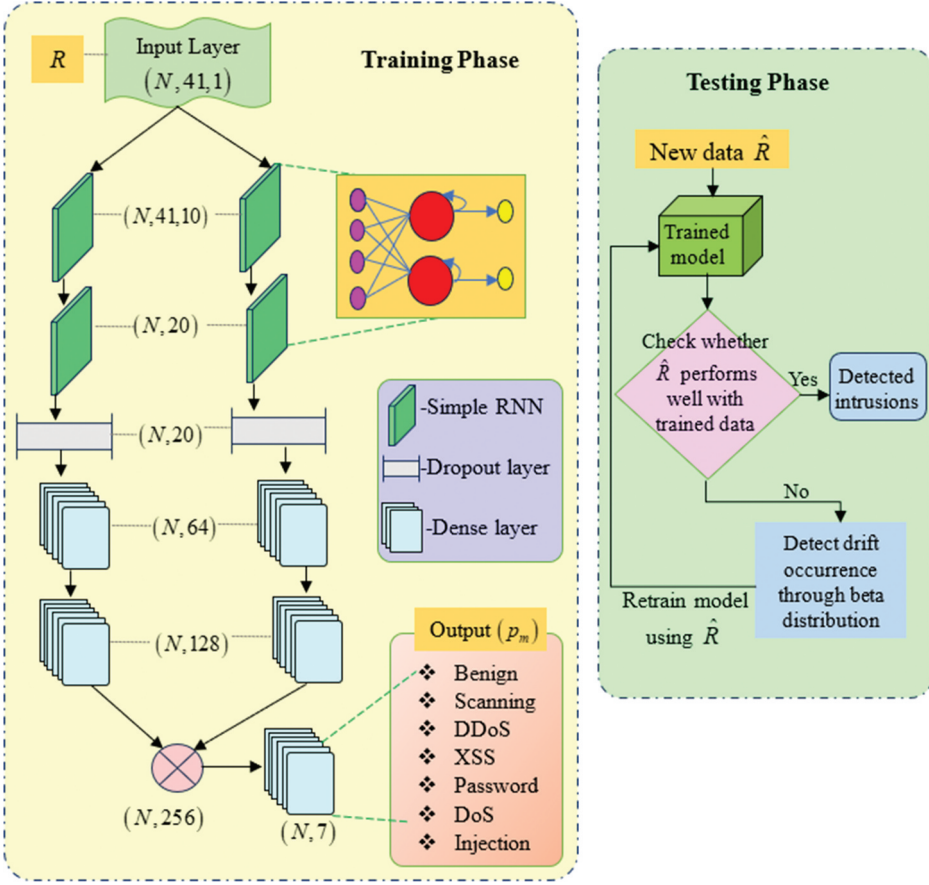


Figure 3. Architecture of the HD²RNet.

modelling probabilities and proportions [30], thus making the model appropriate for drift detection by monitoring the changes in a model's error rate. Initially, a sliding window of length ' N ' is introduced through which the algorithm sets the margin to detect recent data points or samples in the data stream $\hat{R}$. For the predicted probabilities p_m , a confidence score n_u is generated based on the train and test data matches in the range $[0, 1]$. Additionally, the parameters λ and δ are initialised, where λ controls how sensitive the algorithm is to changes in the data distribution. A smaller λ means higher sensitivity to change, which makes the detection more complex and δ determines the padding, which refers to the amount of data left on either side of the sliding window that will not be included in certain parts of the calculations. Then, the sliding window initialised with the test data is divided into two segments such as reference window (l_r) with split up of the first part of test data of confidence scores $[n_1, \dots, n_k]$ and the target window (l_t) with confidence scores $[n_{k+1}, \dots, n_N]$. l_r is the mean of the first segment of data from the beginning of the window to the k^{th} element and l_t is the mean of the second segment of data from the $k + 1^{th}$ element to the N^{th} element. For the following two windows, the mean is computed, which can be denoted as o_r for the target and o_r for the reference window. Then, the mean is compared with the reference and target confidence

scores. The larger deviations in comparisons indicate a higher chance of occurring drift, while lower deviations indicate a lower chance. The possible suspected drift can be checked using the condition *if* $o_r \leq (1 - \lambda)$. If this condition is true, a possible drift is detected, and the algorithm will reset some values and proceed to more detailed analysis, otherwise less chance of drift occurrence. Then, the beta distribution parameters are estimated for both the reference and target window by applying the Maximum Likelihood Estimation (MLE) technique, which can be represented as,

$$g(n_u|\alpha, \beta) = \begin{cases} \frac{n_u^{\alpha-1}(1-n_u)^{\beta-1}}{H(\alpha, \beta)}; & \text{if } 0 < n_u < 1 \\ 0; & \text{otherwise} \end{cases} \quad (9)$$

Next, the beta distribution's Probability Density Function (PDF) is provided by

$$H(\alpha, \beta) = \int_0^1 n_u^{\alpha-1} (1 - n_u)^{\beta-1} dn_u \quad (10)$$

where α , and β implies the distribution parameters, $g(n_u|\alpha, \beta)$ implies the distribution function with confidence score, dn_u implies the differential function of n_u and the $H(\alpha, \beta)$ defines each part of the PDF of the Beta distribution. After this, compute the dissimilarity score between the PDF values of both distributions, which are then aggregated to derive the total dissimilarity score Y_k based on the log-likelihood ratio to confirm the drift occurrence.

$$Y_k \leftarrow Y_k + \log \left(\frac{g(n_u|\alpha_t, \beta_t)}{g(n_u|\alpha_r, \beta_r)} \right) \quad (11)$$

For each point in the test set $u = k + 1$ to N , the log-likelihood ratio is calculated, which determines how likely each test point is under the test vs. reference distribution. The function Y_g is initialised to hold and track the maximum log-likelihood ratio between the reference segment and the test segment. If the current Y_k is larger than the stored Y_g , it means there is a significant change between the two segments, then update Y_g . It reflects how likely it is that the distribution in the test segment is different from the reference segment, indicating drift. The threshold Δ_{th} is initialised, which serves as a parameter to identify data drift more precisely to make final decisions to update the model. It is set by the inverse proportion to the sensitivity parameter λ . If the cumulative dissimilarity score Y_k is greater than the threshold Δ_{th} , the algorithm returns true; else returns false if no drift is detected. After detecting drift, the precise location of the drift is identified by altering the sliding window size and remeasuring the dissimilarity score. If the updated cumulative score Y'_g exceeds, it indicates that the updated divider progressively aligns more closely with the actual drift location. This iterative process repeats until the peak dissimilarity score Y_g is attained, precisely identifying the point of drift. After detecting the location, the particular segments of the data are allowed for the training process to obtain reliable outcomes. Algorithm 1 depicts the working principle of the drift detection mechanism.

*Input : l—Sliding Window, N—Size of window,
 δ — Padding, λ — Sensitivity to Change*

*Output : Drift Occurrence based
on boolean value*

```

 $Y_g \leftarrow 0, \Delta_{th} \leftarrow -\log(\lambda), N \leftarrow |l|$ 
for  $k \leftarrow \delta$  to  $N - \delta$  do
   $o_r \leftarrow \text{mean}(n_1 : n_k \in l)$ ;
   $o_t \leftarrow \text{mean}(n_{k+1} : n_N \in l)$ ;
  if  $o_t \leq (1 - \lambda) \cdot o_r$  then
     $Y_k \leftarrow 0$ 
     $[a_r, \beta_r] \leftarrow \text{estimatebeta} - \text{parameters}(n_1 : n_k)$ ;
     $[a_t, \beta_t] \leftarrow \text{estimatebeta} - \text{parameters}(n_{k+1} : n_N)$ ;
    for  $i \leftarrow k + 1$  to  $N$  do
       $Y_k \leftarrow Y_k + \log\left(\frac{g(n_i|a_t, \beta_t)}{g(n_i|a_r, \beta_r)}\right)$ ;
   $Y_g \leftarrow \max(Y_g, Y_k)$ ;
  if  $Y_g > \Delta_{th}$ 
    return true
  else
    return false

```

Algorithm 1. Drift mechanism using Sliding window-based Beta distribution

A sliding window-based Beta distribution drift detector mechanism is considerably beneficial when dealing with dynamic and changing data patterns, especially in IoT-edge systems where decision-making is essential. It allows a localised view of new data by the sliding window, enabling continuous monitoring of incoming data streams, so that timely detection of changes in network behaviour can be made. This makes it particularly effective in identifying data drift, where the statistical properties of the input data change either slowly or abruptly over time. By integrating the Beta distribution into the model allows for the evaluation of the degree of change between segments of data, where it can identify the uncertainty and variability of observed patterns. The parameters of the Beta distribution are updated dynamically depending on the past and present data so that the system can identify differences between average fluctuations and true drifts. Further, the model gets retrained with new classes formed with the attack patterns based on the concept drift, and hence the hyperparameters are updated with the change in the underlying data distribution. This enhances the sensitivity and accuracy of the drift detecting procedure, decreasing false alarms and, at the same time, ensuring that the significant deviations that portray an intrusion are detected earlier. Overall, this approach improves the adaptability, responsiveness, and reliability of IDS deployed in complicated and resource-limited IoT-edge environments.

5. Results and discussion

This section presents the results and a detailed discussion of the HD²RNet utilised for detecting intrusions.

5.1. Experimental setup

The entire experimentation is carried out on the 64-bit Windows 11 OS utilising the Python Tool encapsulating the packages such as PyTorch, Keras, and TensorFlow. The internal specifications include an Intel Core i7 processor running at 3.2 GHz, 16 GB of RAM, and a ROM of 100GB. Table 1 outlines the hyperparameter settings of the HD²RNet. Further, the hyperparameters are selected based on the preliminary tuning to achieve the optimal performance of intrusion detection.

5.2. Performance measures

The following research incorporates measures like accuracy, recall, precision, and F1-score. Table 2 presents the formulation of these measures.

where A_a implies the positive true instances, A_b implies positive false instances, B_a, B_b implies negative true and negative false instances.

5.3. Dataset description

For validation of the HD²RNet model, the TON-IoT dataset [22] and the BoT-IoT dataset [23] are utilised, and the description of these datasets is mentioned below and the dataset visualisation of both datasets is illustrated in Figure 4.

TON-IoT dataset: The TON-IoT dataset [22] is the extended version of the NetFlow version of the NF-IDS datasets, which has the expansion of features required for network analysis processes. It was collected by the University of Queensland under the comma-separated value (CSV) file, determining both legitimate and malicious behaviours, and determining seven classes of output.

BoT-IoT dataset: The BoT-IoT dataset [23] was generated by the Cyber Range Lab of UNSW, Canberra, which includes network traffic flows and also the normal network

Table 1. Hyperparameter settings.

Hyperparameters	Values
Learning rate	0.01
Layer size	3
Batch size	64
Loss	Categorical Cross-Entropy
Optimiser	Adam
Filter size	128
Kernel size	(1,1)
Activation function	ReLU
Dropout rate	0.5

Table 2. Performance metrics.

Metrics	Formula
Accuracy	$accuracy = \frac{A_a + A_b}{A_a + A_b + B_a + B_b}$
Recall	$recall = \frac{A_a}{A_a + B_b}$
Precision	$precision = \frac{A_a}{A_a + B_a}$
F1-Score	$F1 - score = 2 * \frac{recall \times precision}{recall + precision} \times 100$

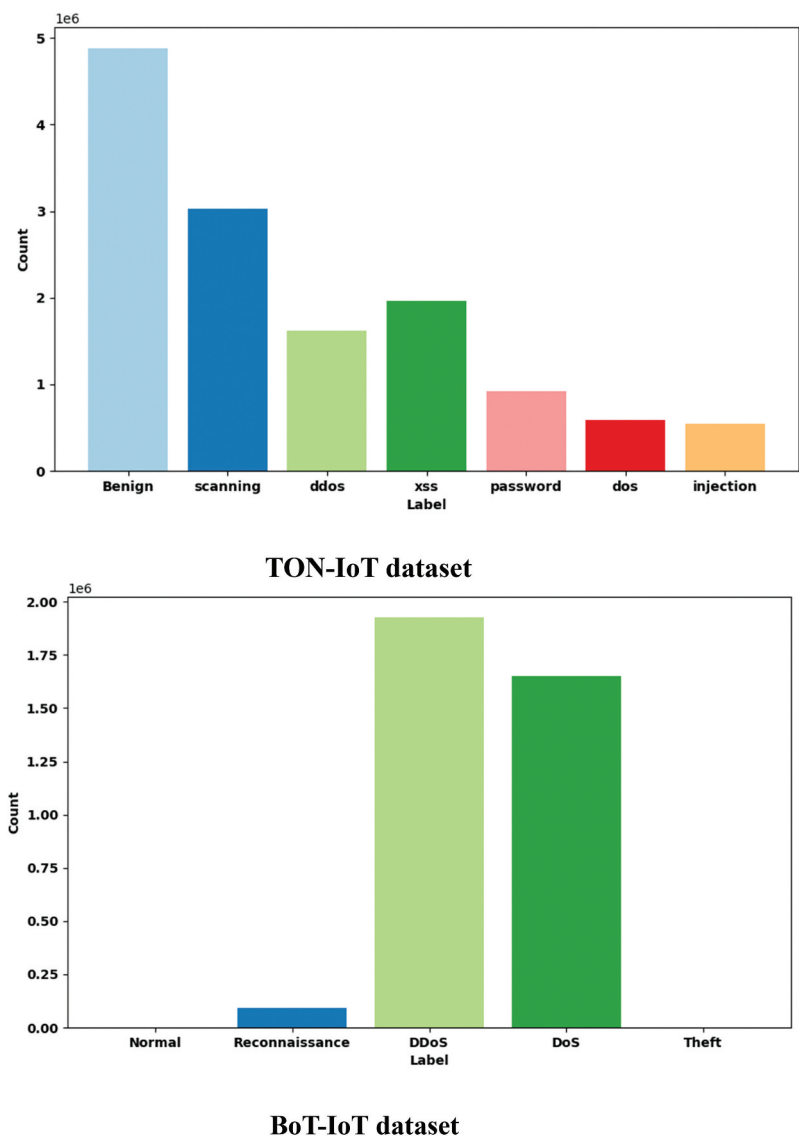


Figure 4. Dataset distribution.

patterns comprising 72,000,000 records in a CSV file. The following database forms a valuable asset to be utilised for attack detection tasks, with a total size of 1.07GB, with five output labels determining emerging intrusions.

CSE-CIC-IDS2018 dataset: The CSE-CIC-IDS2018 dataset [31] is a collaboration project developed in 2018 by CSE and CIC and contains 10 CSV files representing 10 days from the captured network flow, with more than 16.2 million instances. Furthermore, more than 80 features were extracted by the CICFlowMeter and comprised six intrusion attack types such as bot, infiltration, DDoS, DoS, brute force, and web attacks.

5.4. Performance analysis

The performance of the HD²RNet framework is evaluated on both datasets by varying the training percentages and is presented in this section.

5.4.1. Performance evaluation based on the TON-IoT dataset

The performance assessment of the HD²RNet model examined under different training percentages through the TON-IoT dataset is displayed in Figure 5. The performance is analysed under epoch rates of 20 epochs to 100 epochs. The HD²RNet achieved an enhancement at all the successive epochs across all the metrics while varying the training percentage. For 90% training percent, the superior performance of the model can be analysed, where the proposed HD²RNet attained 94.54% accuracy for an epoch count of 20, 95.43% under 40 epochs, 96.42% under 60 epochs, and 97% for 80 epochs, and a superior 97.68% along 100 epochs. In the same way, for other measures, the HD²RNet at 90% gained a superior recall of 97.26%, a precision rate of 98.16%, and a 97.71% F1 score rate showcasing its improved capability in detecting intrusions.

5.4.2. Performance evaluation based on the BoT-IoT dataset

Figure 6 depicts the graphical illustration of the performance outcomes of the BoT-IoT dataset. The HD²RNet, while tested under successive epoch rates ranging from 20 to 100

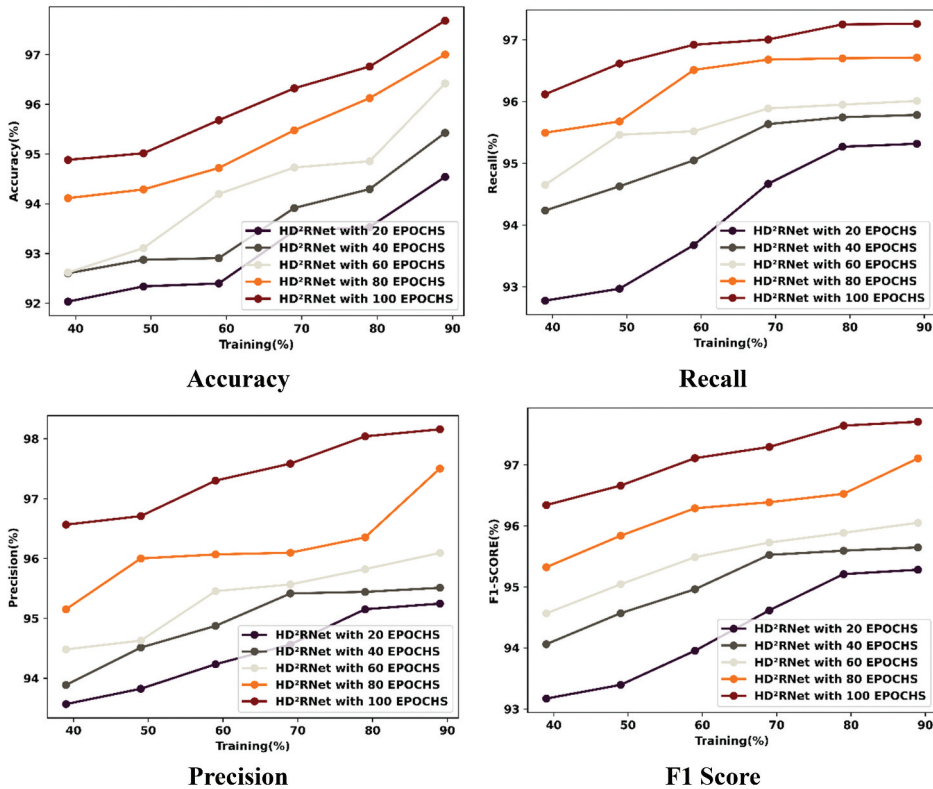


Figure 5. Performance assessment based on the TON-IoT dataset.

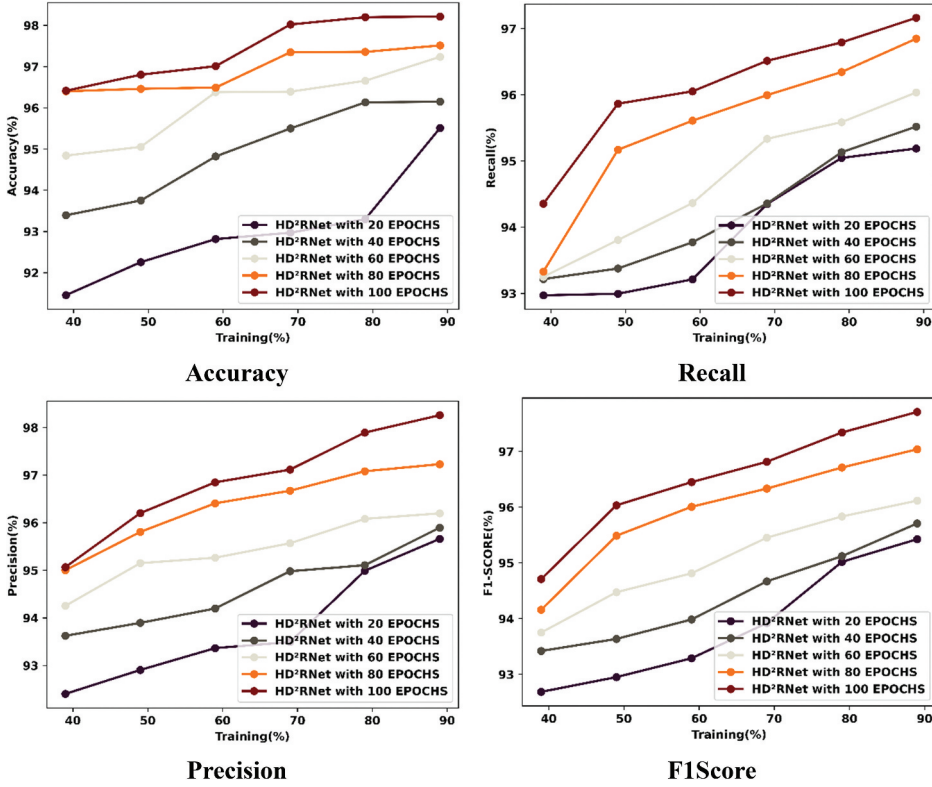


Figure 6. Performance assessment based on the BoT-IoT dataset.

epochs, where the model showed increased performance across the epoch rates, with increments in performance as the training percent increased. The superiority rate of performance achieved by the HD²RNet is at a training percent of 90%, where it attained 98.21% accuracy, 97.16% recall, 98.26% precision, and a 97.71% F1score, replicating its applicability in IoT-edge systems for detecting intrusions more precisely.

5.5. Comparative methods

The proposed HD²RNet model used for detecting intrusion behaviours in IoT-edge based environments is compared against various other methodologies like LDA-LR [2], K-Nearest Neighbour and Random Forest (KNN-RF) [32], DML [3], FL-IDS [1], SEC-IDS [17], BiGRU [18], (Blended IDS) [21], Effective Seeker Optimisation model with Machine Learning-Enabled Intrusion Detection System (ESOML-IDS) [13], SSA-MLP [20], and HCRNN [33] used earlier in the context to determine the efficacy. In order to achieve the fairness of comparison, all the existing models were implemented under the same experimental conditions (same dataset, hardware, and preprocessing) used in the proposed method.

5.5.1. Comparative evaluation based on the TON-IoT dataset

The comparative results analysed on the TON-IoT dataset with differing training percentages are shown in Figure 7. The HD²RNet attained the highest rate of accuracy, 97.68%, at

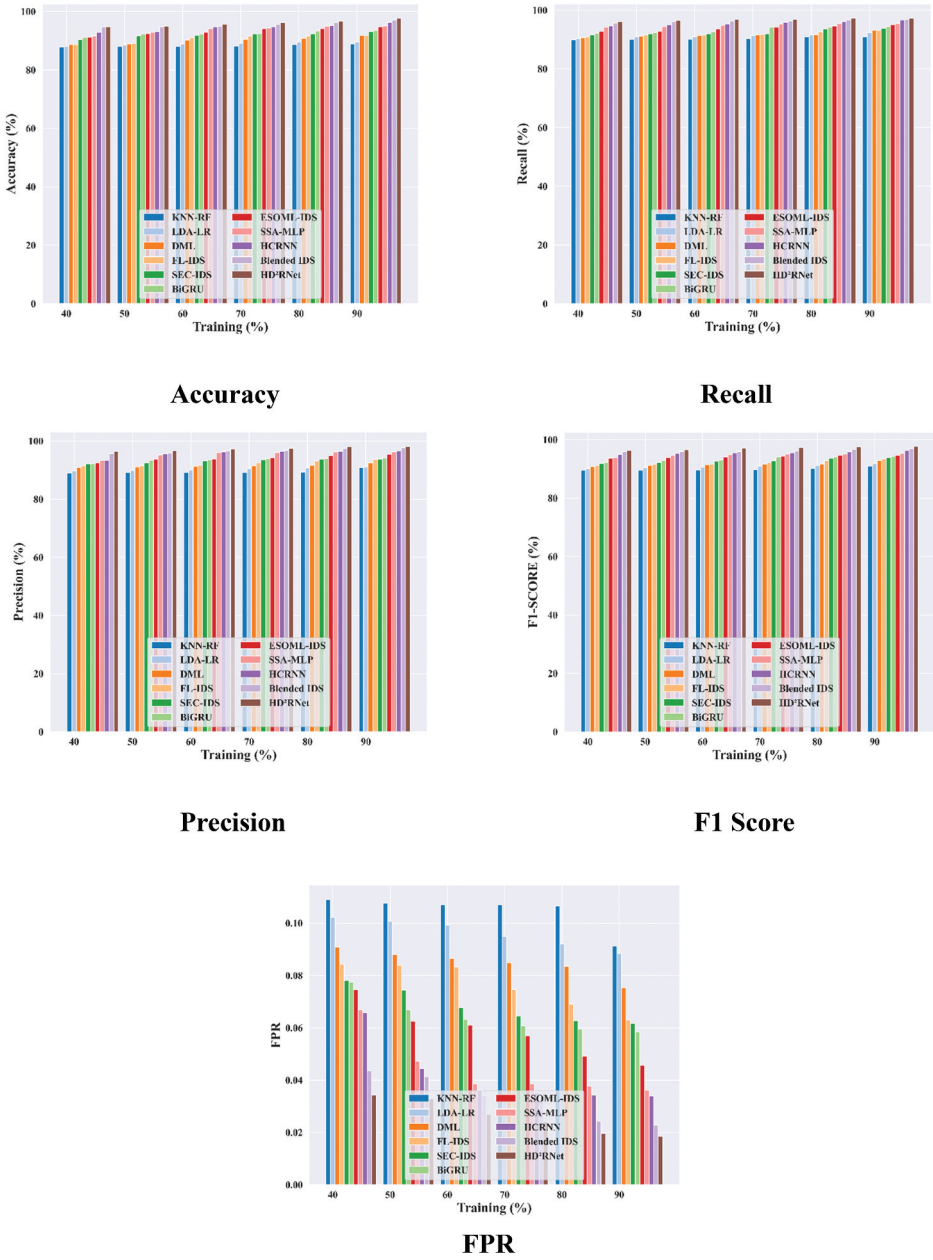


Figure 7. Comparison analysis based on the TON-IoT dataset.

a training percent 90%, while the KNN-RF, LDA-LR, DML, FL-IDS, SEC-IDS, and BiGRU attained 97.68%, 89.62%, 91.78%, 91.83%, 93.18%, and 93.51%, which shows an improvement rate of 8.89% over KNN-RF, 8.25% over LDA-LR, 6.04% over DML, 5.99% over FL-IDS, 4.61% over SEC-IDS, and 4.27% over BiGRU. Similarly, for other metrics, the marginal improvements in performance rate are replicated where the proposed HD²RNet model reported a highest recall of 97.26%, precision of 98.16%, and 97.71% F1 score for the 90% of training, respectively.

5.5.2. Comparative evaluation based on the BoT-IoT dataset

In this section, Figure 8 portrays the comparative analysis of the HD²RNet analysed on the BoT-IoT dataset along with different training percentages. The proposed HD²RNet showcased an enhancement in performance across all the metrics, effectively overtaking other models utilised for comparison in terms of performance. While training the model at the highest training rate 90%, the HD²RNet reported a high accuracy of 98.21%, significantly surpassing existing methods such as KNN-RF by 10.01%, LDA-LR by 7.96%, DML by 7.61%, FL-IDS by 6.19%, SEC-IDS by 4.48%, and BiGRU by 3.47%. Similarly, for other metrics, successive marginal differences can be seen in performance with the proposed HD²RNet. The HD²RNet at 90% gained the highest value of 97.16% for recall, 98.26% for precision, and 97.71% F1 score, which is greater than other methods.

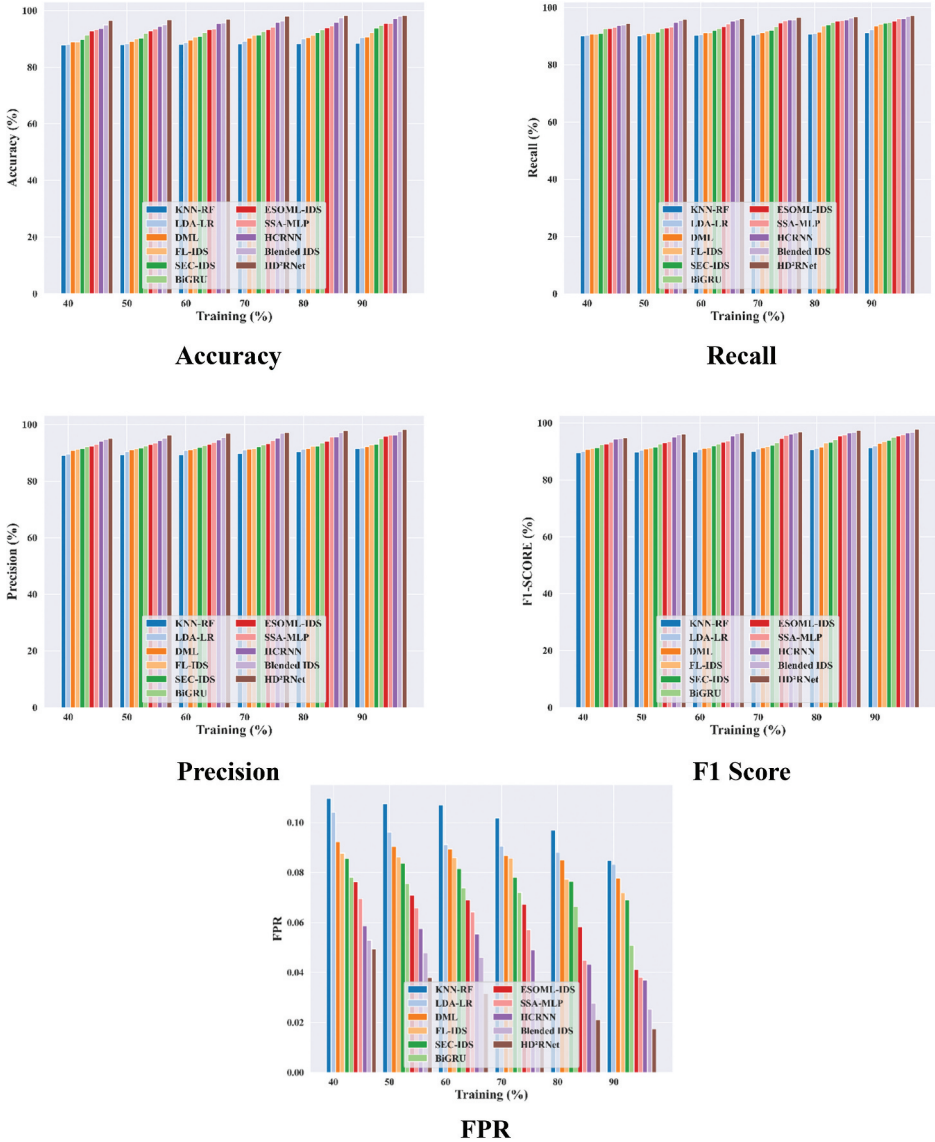


Figure 8. Comparison analysis based on the BoT-IoT dataset.

5.5.3. Comparative evaluation based on the CSE-CIC-IDS 2018 dataset

In this section, the results attained with the proposed model are compared with the existing methods by varying the proportion of training data from CSE-CIC-IDS 2018 dataset, and the visualisation of results is presented in Figure 9. Notably, the proposed HD²RNet reported the high accuracy of 99.28% with 90% of training, indicating the improvement margin of 3.71% over KNN-RF, 3.39% over LDA-LR, 1.69% over ESOML-

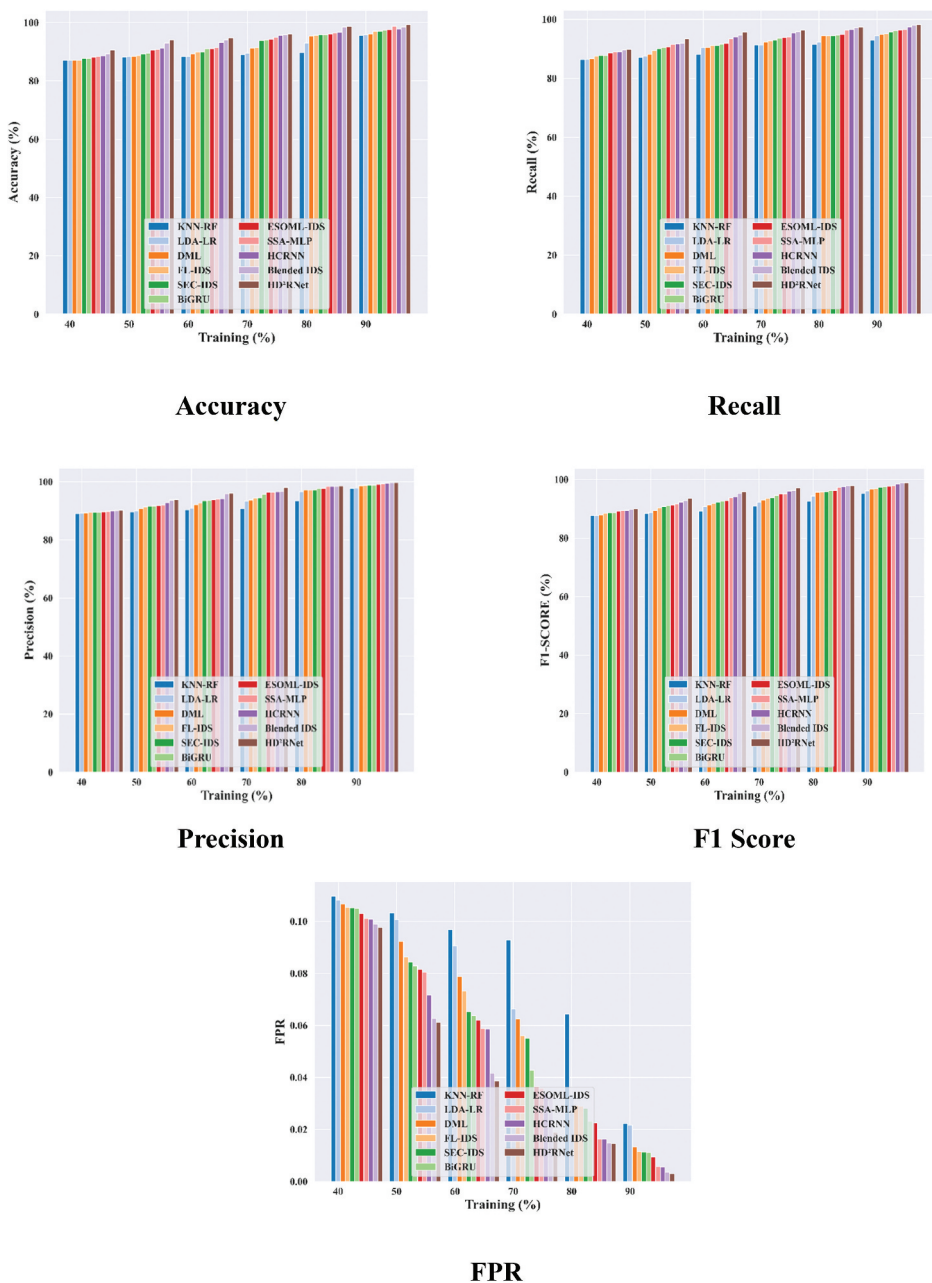


Figure 9. Comparative evaluation based on the CSE-CIC-IDS 2018 dataset.

IDS, 1.53% over HCRNN, and 0.75% over Blended IDS. Subsequently, the proposed HD²RNet obtained a high recall of 98.16% for 90% of training, outperforming the existing methods with a substantial improvement margin of 5.28% against KNN-RF, 3.74% against LDA-LR, 1.84% against ESOML-IDS, 1.68% against SSA-MLP, and 0.70% against HCRNN. Regarding the metric precision, the proposed HD²RNet surpassed the existing models and shows the performance improvement of 1.93% with KNN-RF, 1.87% with LDA-LR, 0.63% with ESOML-IDS, 0.27% with SSA-MLP, and 0.25% with HCRNN. Comparatively, the proposed HD²RNet achieved a high F1-score of 98.93%, exceeding the conventional methods ESOML-IDS, SSA-MLP, HCRNN, and Blended IDS by 1.24%, 0.98%, 0.48%, and 0.08%, respectively. In addition, the proposed HD²RNet significantly reduced the FPR rate to 0.003, indicating the difference of 0.010 against KNN-RF, 0.008 against LDA-LR, and 0.006 against ESOML-IDS. Moreover, the findings of the comparison reveal that the proposed HD²RNet based intrusion detection system consistently outperforms the other state-of-the-art methods utilised for the evaluation. Specifically, the proposed method integrates the benefits of the distributed weight-tuned RNN in extracting the distinctive features and the drift detection ability to capture the dynamic changes or attack patterns in the network traffic, contributing to enhanced intrusion detection.

5.6. Statistical analysis

Statistical analysis helps to analyse the deviations and patterns of the outcomes quantitatively. The statistical outcomes of the HD²RNet against other existing methods are analysed, based on the statistical measures such as best, mean, and variance, evaluated across different training scales, and are displayed in [Table 3](#).

5.7. Confusion matrix

The Confusion matrix is utilised to determine the prediction performance by analysing how well the predicted labels were accurately classified and where the model replicates errors. The confusion matrix obtained under the predicted labels of both the TON-IoT dataset and the BoT-IoT dataset is depicted in [Figure 10](#).

5.8. ROC analysis

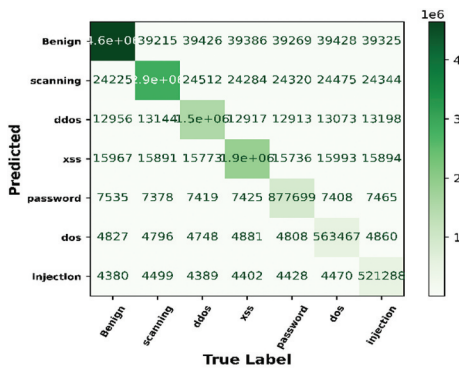
ROC helps to measure the performance rate of the classification system to determine how well the framework predicts the distinguished labels accurately and compares the AUC. In the context of intrusion detection, True Positive Rate (TPR) signifies the sensitivity that evaluates the proportion of real positive instances correctly detected by the detection model. Contrastingly, the False-Positive Rate (FPR) indicates the proportion of real negative cases that are incorrectly detected as positive [Figure 11](#). illustrates the ROC analysis of the proposed method and existing methods.

5.9. Computation complexity analysis

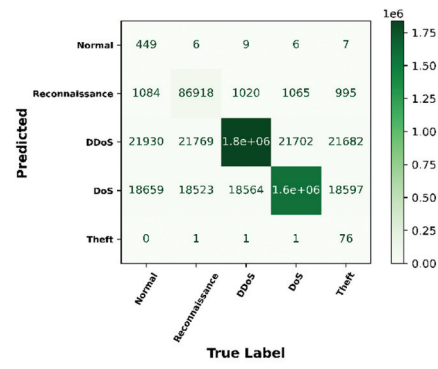
[Figure 12](#) implies the computational time analysis of the HD²RNet model through a graphical representation. For epoch 100, the proposed HD²RNet takes 20.57s for

Table 3. Statistical results of the HD²RNet with existing approaches.

		Statistical Results based on TON-IoT dataset						
Methods		KNN-RF	LDA-LR	DML	FL-IDS	SEC-IDS	BiGRU	HD ² RNet
Mean	Accuracy (%)	88.32	88.95	90.17	90.62	91.98	92.50	96.06
	Recall (%)	90.39	91.29	91.65	91.97	92.52	93.39	96.86
	Precision (%)	89.52	90.37	91.51	92.36	93.18	93.56	97.39
	F1 Score (%)	89.95	90.82	91.58	92.16	92.85	93.47	97.12
Best	Accuracy (%)	89.00	89.62	91.78	91.83	93.18	93.51	97.68
	Recall (%)	91.01	92.36	93.25	93.28	93.93	94.56	97.26
	Precision (%)	90.87	91.16	92.46	93.70	93.83	94.15	98.16
	F1 Score (%)	90.94	91.76	92.85	93.49	93.88	94.35	97.71
Variance	Accuracy (%)	0.15	0.32	1.15	1.56	0.75	0.61	0.97
	Recall (%)	0.17	0.36	0.63	0.62	0.86	0.91	0.16
	Precision (%)	0.37	0.25	0.24	0.67	0.37	0.42	0.37
	F1 Score (%)	0.23	0.30	0.41	0.63	0.53	0.59	0.24
Methods		Statistical Results based on the BoT-IoT dataset						
		KNN-RF	LDA-LR	DML	FL-IDS	SEC-IDS	BiGRU	HD ² RNet
Mean	Accuracy (%)	88.11	89.06	89.83	90.69	91.36	92.66	97.44
	Recall (%)	90.44	90.86	91.45	92.08	92.47	93.45	96.12
	Precision (%)	89.87	90.78	91.31	91.75	92.10	93.05	96.90
	F1 Score (%)	90.16	90.82	91.38	91.91	92.28	93.25	96.51
Best	Accuracy (%)	88.38	90.39	90.74	92.13	93.81	94.80	98.21
	Recall (%)	91.17	92.28	93.51	94.24	94.47	94.84	97.16
	Precision (%)	91.52	91.68	92.24	92.81	93.10	94.91	98.26
	F1 Score (%)	91.34	91.98	92.87	93.52	93.78	94.88	97.71
Variance	Accuracy (%)	0.04	0.78	0.52	1.08	1.80	1.24	0.52
	Recall (%)	0.15	0.44	0.91	1.81	1.61	0.98	0.81
	Precision (%)	0.72	0.43	0.23	0.33	0.30	0.82	1.12
	F1 Score (%)	0.38	0.38	0.50	0.91	0.80	0.84	0.95



TON-IoT dataset



BoT-IoT dataset

Figure 10. Confusion matrix.

execution, while the KNN-RF required 20.82s, LDA-LR required 20.6s, DML required 20.71s, FL-IDS required 20.74s, SEC-IDS required 20.75s, and BiGRU required 20.77s for execution, which is significantly shorter than existing approaches. Specifically, the proposed method utilises the distributed learning mechanism, which highly contributes to reducing the computational complexity compared to other existing methods used for the evaluation.

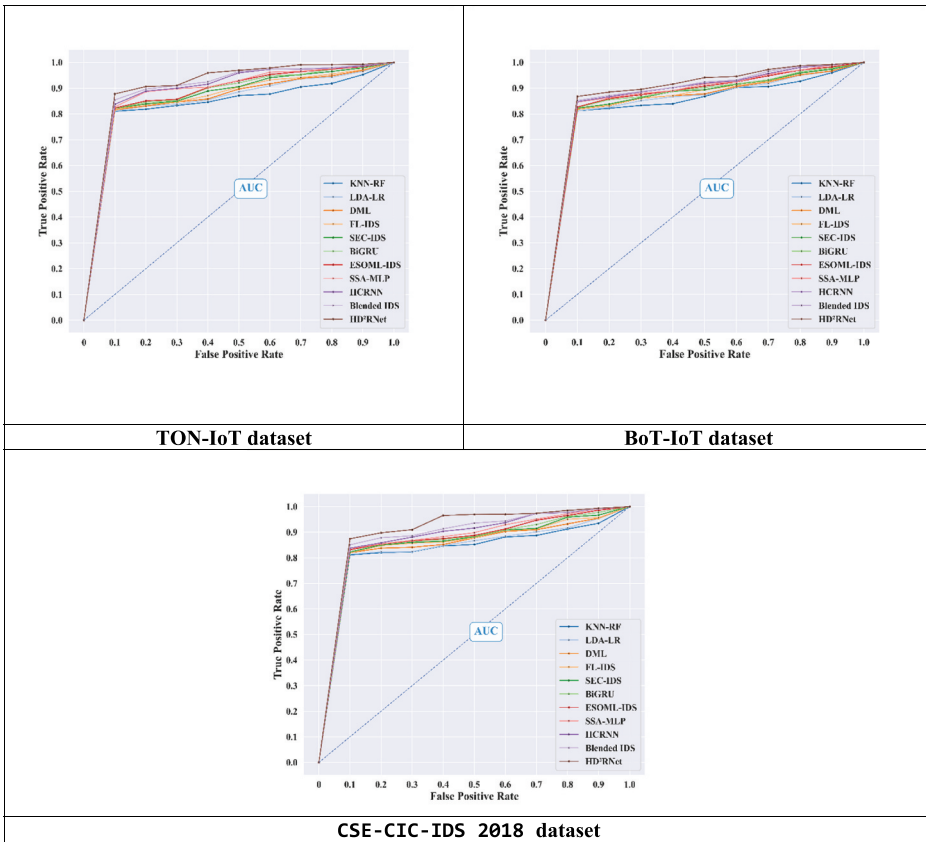


Figure 11. ROC analysis.

5.10. Comparative discussion

The rapid growth of IoT-based systems possesses an enormous amount of data, requiring advanced systems for reliable data processing. Hence, edge computing is introduced, which forms an important paradigm that processes data with lower latency and faster. However, the resource constraints and the decentralised mode of communication make the process tedious by producing security concerns and necessitating the requirement for a more secure and efficient way to tackle network traffic flow. The traditional approaches, like KNN-RF [32], incorporated machine learning algorithms to detect the intrusions; however, the increased rate of false alarms obscured the detection process and led to overfitting problems. The LDA-LR [2] utilised regression strategies to understand the patterns of cyber data; however, the integration of time-series analysis limited the model's ability to detect various types of attacks, leading to scalability and resource-constrained issues. The DML [3] model incorporated sophisticated ML frameworks but failed to model sequential dependencies of the network patterns that change over time, producing abnormal deviations in performance rates. While the FL-IDS [1] and the SEC-IDS [17] approach utilises two different strategies, such as federated learning along with deep learning classifiers to understand and detect emerging intrusions. However, the

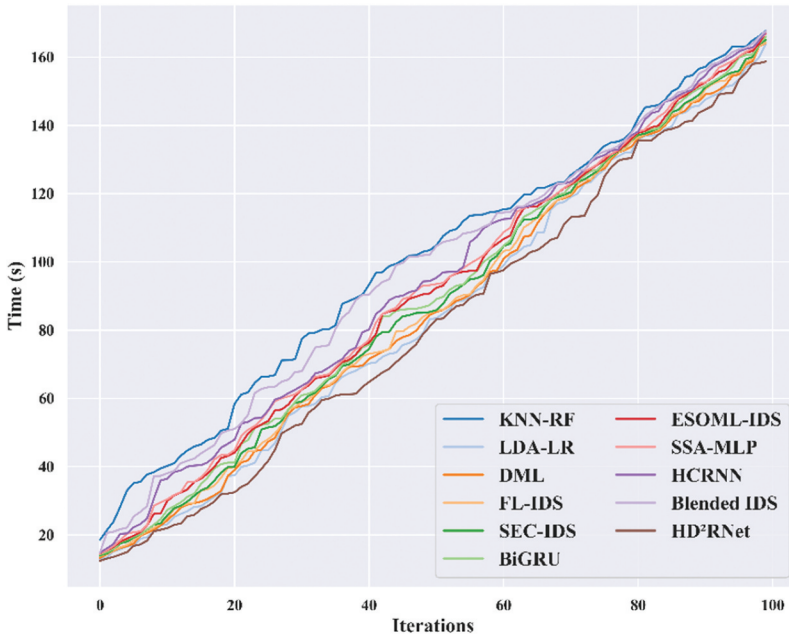


Figure 12. Analysis of computational complexity.

insufficiency of efficient data for classification limits its applicability, and maintaining a reliable performance remains a significant challenge. While the BiGRU [18] trained upon gated units produced an enlarged computational burden and adaptability challenges due to the distributed architecture. These challenges are effectively addressed by the proposed HD²RNet model by understanding the network patterns effectively through sequential processing of features in parallel through a distributed architecture. Specifically, the proposed models utilise the drift threshold to analyse the data distribution and detect the attack patterns that are above the threshold value and operate by maintaining a sliding window of the recent data points and observing the data distribution varying over time. On detecting the distribution changes significantly within the window, the drift detection signals the occurrence of a concept drift. Additionally, the drift concept enhances the model's adaptiveness against data distributions, empowers the IDS to effectively identify previously unknown threats, and considers those attack patterns as new classes and re-trains the model for intrusion detection. The performance outcomes replicate the efficacy of the model by gaining significant improvements over other approaches. The comparison assessment of the proposed HD²RNet with existing methods based on 90% of training using TON-IoT dataset, BoT-IoT dataset, and CSE-CIC-IDS 2018 dataset is demonstrated in Tables 4–6, respectively.

5.11. Ablation study

Ablation study is conducted to highlight the significance of the different components involved in the proposed architecture, and the visualisation of the results is presented in

Table 4. Comparative discussion of HD²RNet based on TON-IoT dataset.

Methods	Training percentage-90				
	Accuracy (%)	Recall (%)	Precision (%)	F1-score (%)	FPR
KNN-RF	89.00	91.01	90.87	90.94	0.091
LDA-LR	89.62	92.36	91.16	91.76	0.088
DML	91.78	93.25	92.46	92.85	0.075
FL-IDS	91.83	93.28	93.70	93.49	0.062
SEC-IDS	93.18	93.93	93.83	93.88	0.061
BiGRU	93.51	94.56	94.15	94.35	0.058
ESOML-IDS	94.81	95.08	95.42	94.77	0.045
SSA-MLP	95.10	95.59	96.38	95.30	0.036
HCRNN	96.34	96.77	96.60	96.39	0.033
Blended IDS	97.07	96.86	97.72	96.93	0.022
HD ² RNet	97.68	97.26	98.16	97.71	0.018

Table 5. Comparative discussion of HD²RNet based on BoT-IoT dataset.

Methods	Training percentage-90				
	Accuracy (%)	Recall (%)	Precision (%)	F1-score (%)	FPR
KNN-RF	88.38	91.17	91.52	91.34	0.084
LDA-LR	90.39	92.28	91.68	91.98	0.083
DML	90.74	93.51	92.24	92.87	0.077
FL-IDS	92.13	94.24	92.81	93.52	0.071
SEC-IDS	93.81	94.47	93.10	93.78	0.069
BiGRU	94.80	94.84	94.91	94.88	0.050
ESOML-IDS	95.50	95.23	95.89	95.46	0.041
SSA-MLP	95.50	96.01	96.19	95.82	0.038
HCRNN	97.13	96.15	96.31	96.53	0.036
Blended IDS	98.07	96.87	97.47	96.72	0.025
HD ² RNet	98.21	97.16	98.26	97.71	0.017

Table 6. Comparative discussion of HD²RNet based on CSE-CIC-IDS 2018 dataset.

Methods	Training percentage-90				
	Accuracy (%)	Recall (%)	Precision (%)	F1-score (%)	FPR
KNN-RF	95.59	92.98	97.77	95.31	0.022
LDA-LR	95.90	94.49	97.83	96.13	0.021
DML	96.01	94.82	98.67	96.71	0.013
FL-IDS	97.04	95.04	98.85	96.91	0.011
SEC-IDS	97.05	95.75	98.87	97.29	0.011
BiGRU	97.36	96.14	98.88	97.49	0.011
ESOML-IDS	97.59	96.35	99.06	97.69	0.009
SSA-MLP	98.79	96.51	99.43	97.95	0.005
HCRNN	97.75	97.47	99.45	98.45	0.005
Blended IDS	98.53	98.07	99.63	98.84	0.004
HD ² RNet	99.28	98.16	99.70	98.93	0.003

Figure 13. In the ablation experiment, specific components of the proposed architecture are eliminated to assess the impact of each component in boosting the intrusion detection performance. The experimental findings show that the proposed model without distributed learning demonstrates a significant drop in performance, attaining a reduced accuracy of 96.56%. Additionally, the proposed model without drift detection module reported a low accuracy of 95.57% for intrusion detection. Subsequently, the proposed model without a dropout layer attains a reduced accuracy of 98.46%. Consequently, the ablation study highlights the significant contribution of the drift detection module in

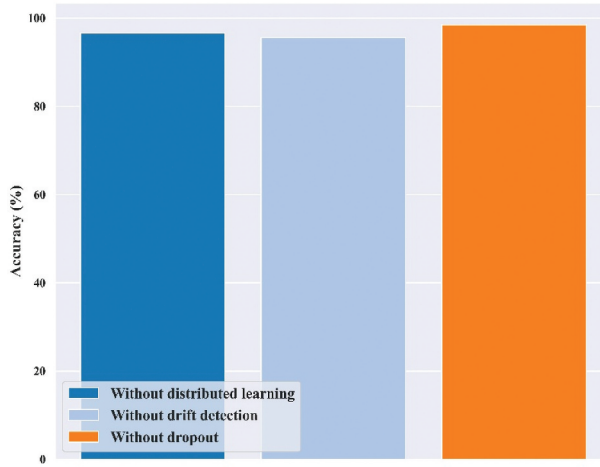


Figure 13. Ablation study.

identifying the dynamic changes caused by the attack in network traffic. Overall, the ablation study demonstrates the significant contribution of the different modules integrated in the proposed model for intrusion detection.

5.12. Computational cost analysis

The comparison of the computational cost between the HD² RNet model and other existing methods is performed over several iterations to demonstrate the effectiveness of the HD² RNet model. The computational complexity with Big O notation demonstrates the order of growth of the model's resource consumption with increasing number of iterations. From the overall computational cost analysis, the proposed method shows reduced computational cost relative to other existing methods. For iteration 100, the HD² RNet model exhibits the least computational complexity of $O(1)$ in comparison to other existing methods. Specifically, the incorporation of a distributed learning mechanism significantly contributed to reducing the computational complexity of the proposed model. Figure 14 visualises the computational complexity analysed in terms of Big O notation for the proposed method and existing methods.

5.13. Sensitivity analysis

Sensitivity analysis is conducted to explore the performance of the HD² RNet model by varying the range of input parameters. In this section, the sensitivity analysis evaluates how the proposed model outputs change when input parameters, including drift thresholds and window sizes, are slightly varied. The sensitivity levels are indicated as 7.48×10^{190} , and 3.214×10^{191} when the sliding window is altered from 1 to 2, and from 1 to 3, respectively. Figure 15 depicts the sensitivity analysis on varying the sliding window.

Figure 16 depicts the sensitivity analysis on varying the drift threshold. Additionally, the sensitivity levels are reported as 1.22×10^{177} , when adjusting the drift threshold from

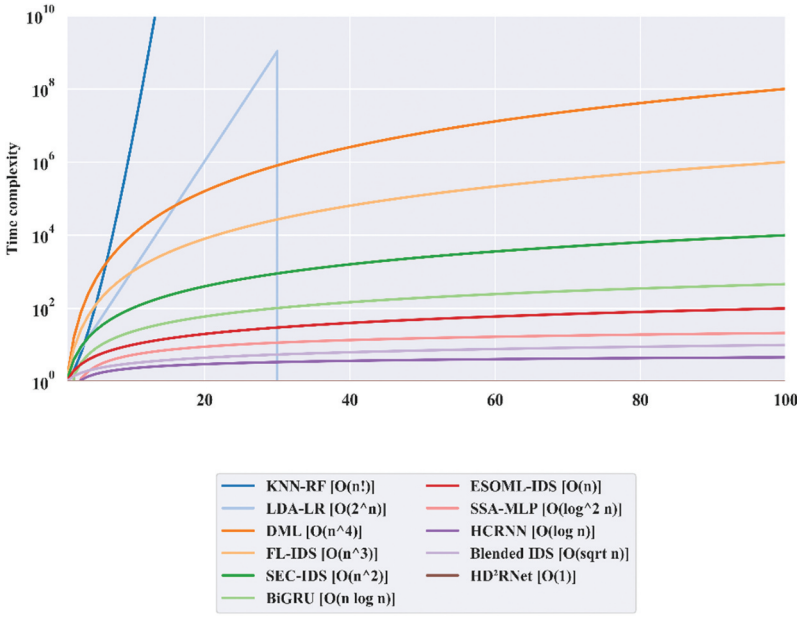
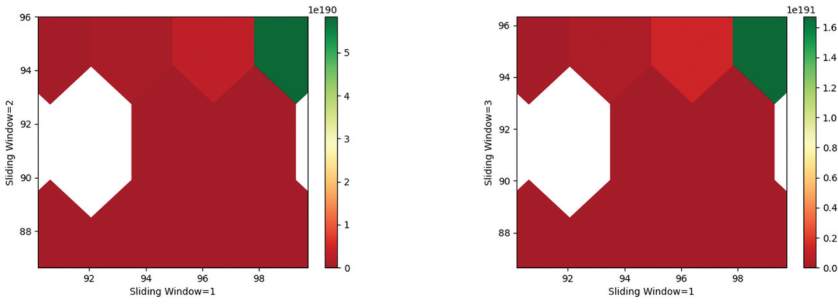


Figure 14. Computational cost analysis.



a) varying the sliding window from 1 to 2 b) varying the sliding window from 1 to 3

Figure 15. Sensitivity analysis on varying the sliding window.

0.1 to 0.2, and reports a sensitivity level of 2.12×10^{178} on varying the drift threshold from 0.1 to 0.3, reports a sensitivity of 4.23×10^{182} on varying the drift threshold from 0.1 to 0.4, and finally reports a sensitivity of 4.23×10^{182} on varying the drift threshold from 0.1 to 0.5.

5.14. Memory usage analysis

In this section, [Figure 17](#) portrays the comparison of the proposed model with conventional models in terms of memory utilisation. The results of the experimental analysis reveal that the memory utilisation of the HD² RNet model is significantly reduced by up to 3.91 MB for accomplishing the intrusion detection. Contrastingly, the other baseline

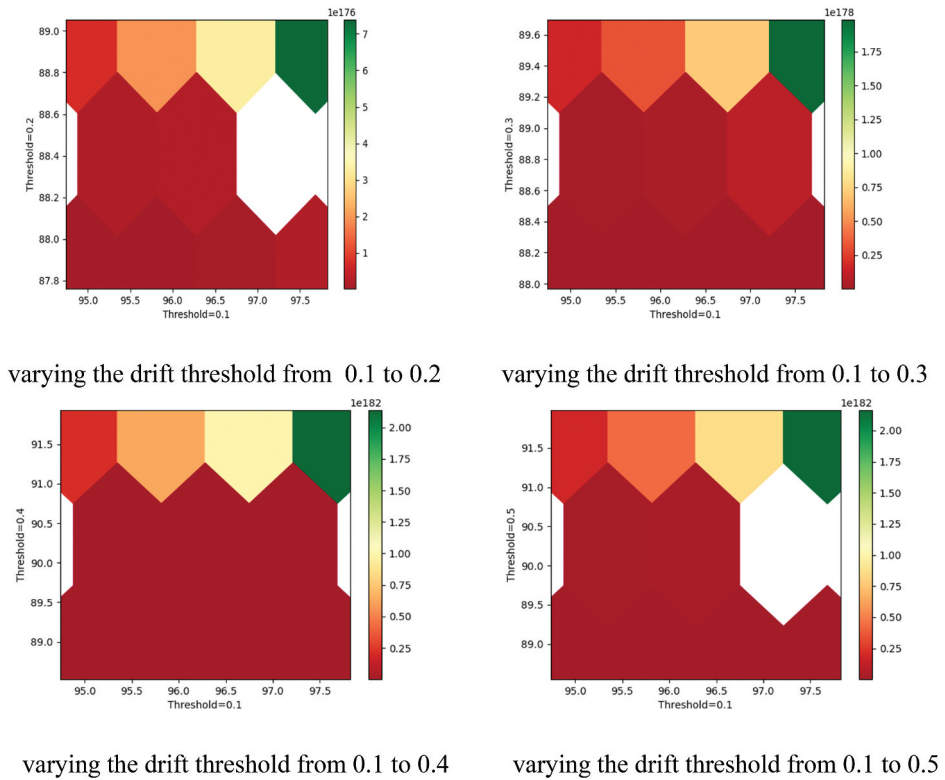


Figure 16. Sensitivity analysis on varying the drift threshold.

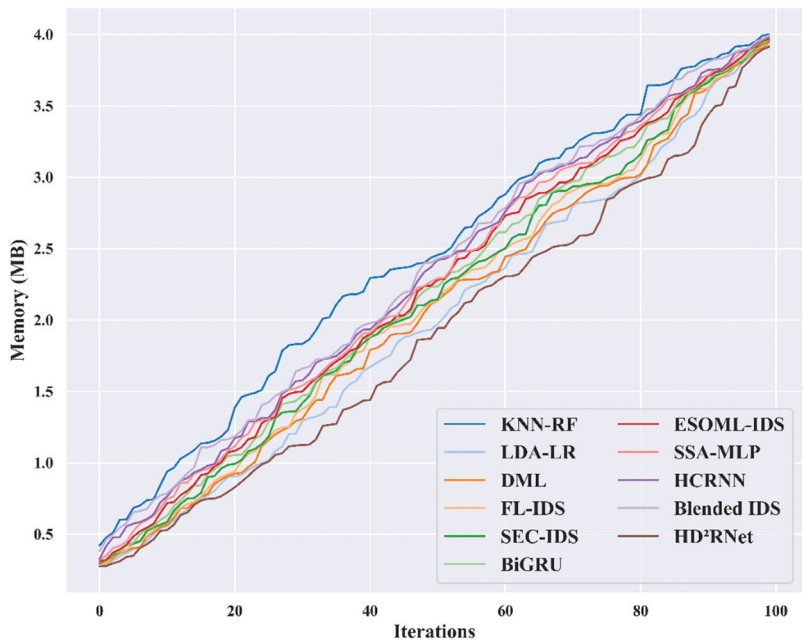


Figure 17. Memory usage analysis.

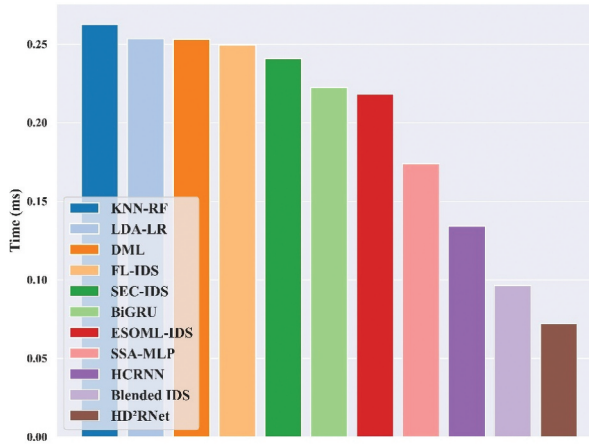


Figure 18. Analysis based on detection delay.

methods highly increased the memory utilisation due to the lack of sophisticated mechanisms. The results indicate that the proposed model reduced the memory consumption, indicating the difference in memory usage of upto 0.08 MB compared to KNN-RF, 0.007 MB compared to LDA-LR, 0.032 MB compared to DML, 0.04 MB compared to FL-IDS, 0.040 MB compared to SEC-IDS, 0.053 MB compared to BiGRU, 0.064 MB compared to ESOML-IDS, 0.066 MB compared to SSA-MLP, 0.081 MB compared to HCRNN. In the proposed method, the application of the distributed learning mechanism significantly reduced the memory consumption through parallel processing of input data. Moreover, the memory usage analysis shows the reduced resource utilisation of the HD² RNet model for its deployment in real-world applications for intrusion detection.

5.15. Analysis of detection delay

In this section, the delay reported by the proposed model and other existing models for completing the task of intrusion detection is analysed in detail. The results obtained with the experimental analysis indicate that the proposed HD² RNet model significantly reduced the detection delay to 0.072 ms. In contrast, the existing methods increased the delay for intrusion detection compared to the proposed model, showing a delay difference of 0.19 ms with KNN-RF, 0.181 ms with LDA-LR, 0.177 ms with DML, 0.168 ms with FL-IDS, 0.150 ms with SEC-IDS, 0.146 ms with BiGRU, 0.101 ms with ESOML-IDS, 0.06 ms with SSA-MLP, 0.024 ms with HCRNN, and 0.02 ms with Blended IDS. The experimental results indicate that the proposed method has reduced the detection delay, compared with recent intrusion detection methods. Figure 18 visualises the analysis based on the detection delay.

6. Conclusion

In this research, an HD² RNet framework is proposed to address the increased demands of designing intrusion detection in IoT-edge infrastructure to ensure secure data processing.

The proposed model leverages sequential learning through an RNN in a distributed manner to efficiently capture the temporal patterns in the network traffic data, which makes it possible to identify anomalous behaviours precisely. The integration of a drift-detection mechanism based on a sliding window technique enabled beta distribution parameters further contribute to enhancing the adaptability to changing network traffic flow, making the model respond to data distribution changes more accurately. Furthermore, the distributed architecture supports parallel and efficient processing on the edge nodes, enabling it as a robust framework for intelligent, adaptive, and context-aware intrusion detection in highly dynamic and resource-limited edge computing environments. During performance evaluation, the HD²RNet obtained a 97.68% accuracy rate, a recall of 97.26%, 98.16% precision, and an F1 score of 97.71% when tested on the TON-IoT dataset with a 90% training rate, significantly outperforming the conventional approaches. However, the real-time systems require operating on networks with limited bandwidth that hinders the seamless exchange of information, significantly causing delays in data transfer and processing crucial for effective intrusion detection. The future work will explore incorporating hybrid optimisation techniques to enhance the effectiveness of training and implementing attention mechanisms to prioritise important information, and enhancing features will be an added advantage, especially in dynamic IoT-edge-based systems.

Acknowledgments

I would like to express my very great appreciation to the co-authors of this manuscript for their valuable and constructive suggestions during the planning and development of this research work.

Author contribution

All authors have made substantial contributions to conception and design, revising the manuscript, and the final approval of the version to be published. Also, all authors agreed to be accountable for all aspects of the work in ensuring that questions related to the accuracy or integrity of any part of the work are appropriately investigated and resolved.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Data availability statement

The data underlying this article are available in TON-IoT dataset, '<https://www.kaggle.com/datasets/dhoogla/nftoniotv2>', and BOT-IoT dataset, '<https://research.unsw.edu.au/projects/bot-iot-dataset>'.

References

- [1] Bhavsar MH, Bekele YB, Roy K, et al. FI-ids: federated learning-based intrusion detection system using edge devices for transportation IoT. IEEE Access. 2024;12:52215–52226. doi: 10.1109/ACCESS.2024.3386631

- [2] Mahadevappa P, Murugesan RK, Al-Amri R, et al. A secure edge computing model using machine learning and IDS to detect and isolate intruders. *MethodsX*. 2024;12:102597. doi: [10.1016/j.mex.2024.102597](https://doi.org/10.1016/j.mex.2024.102597)
- [3] Hoang TM, Le Thi TL, Quy NM. A novel distributed machine learning model to detect attacks on edge computing network. *J Adv Inf Technol*. 2023;14(1):153–159.
- [4] Zeng GQ, Yang YW, Lu KD, et al. Evolutionary adversarial autoencoder for unsupervised anomaly detection of industrial internet of things. *IEEE Trans Rel*. 2025;74(3):3454–3468. doi: [10.1109/TR.2025.3528256](https://doi.org/10.1109/TR.2025.3528256)
- [5] Aldaej A, Ullah I, Ahanger TA, et al. Ensemble technique of intrusion detection for IoT-edge platform. *Sci Rep*. 2024;14(1):11703. doi: [10.1038/s41598-024-62435-y](https://doi.org/10.1038/s41598-024-62435-y)
- [6] Lu KD, Yang YW, Zeng GQ, et al. BPSO-AHDL-IDS: binary particle swarm optimization-based automated hybrid deep learning model for intrusion detection of internet of things. *IEEE Trans Automat Sci Eng*. 2025;22:15859–15877. doi: [10.1109/TASE.2025.3572510](https://doi.org/10.1109/TASE.2025.3572510)
- [7] Hossain MA, Hossain MS, Karim R. Comprehensive architectural network design based on intrusion detection system. *Int J Commun Inf Technol*. 2023;4(2):12–19. doi: [10.33545/2707661X.2023.v4.i2a.66](https://doi.org/10.33545/2707661X.2023.v4.i2a.66)
- [8] Biju A, Franklin SW. Network intrusion detection system with an edge based hybrid feature selection approach. *Computing*. 2024;9:10.
- [9] Fenanir S, Semchedine F. Smart intrusion detection in IoT edge computing using federated learning. *Revue d'Intel Artificielle*. 2023;37(5):1133. doi: [10.18280/ria.370505](https://doi.org/10.18280/ria.370505)
- [10] Bhardwaj A, Tyagi R, Sharma N, et al. Network intrusion detection in software defined networking with self-organized constraint-based intelligent learning framework. *Meas: Sensors*. 2022;24:100580. doi: [10.1016/j.measen.2022.100580](https://doi.org/10.1016/j.measen.2022.100580)
- [11] Kipongo J, Swart TG, Esenogho E. Artificial intelligence-based intrusion detection and prevention in edge-assisted SDWSN with modified honeycomb structure. *IEEE Access*. 2023;12:3140–3175. doi: [10.1109/ACCESS.2023.3347778](https://doi.org/10.1109/ACCESS.2023.3347778)
- [12] Alotaibi NS, Ahmed HI, Kamel SOM. Dynamic adaptation attack detection model for a distributed multi-access edge computing smart city. *Sensors*. 2023;23(16):7135. doi: [10.3390/s23167135](https://doi.org/10.3390/s23167135)
- [13] Alzubi OA, Alzubi JA, Alazab M, et al. Optimized machine learning-based intrusion detection system for fog and edge computing environment. *Electronics*. 2022;11(19):3007. doi: [10.3390/electronics11193007](https://doi.org/10.3390/electronics11193007)
- [14] Kumar A, Ahuja DS, Gupta DG. Edge computing-based intrusion detection system for wireless sensor networks utilizing deep learning algorithms. *SSRN Electron J*. 2024. Available at SSRN 5038712. doi: [10.2139/ssrn.5038712](https://doi.org/10.2139/ssrn.5038712)
- [15] Mahadik SS, Pawar PM, Muthalagu R. Edge-federated learning based intelligent intrusion detection system for heterogeneous Internet of Things. *IEEE Access*; 2024.
- [16] Alladi T, Kohli V, Chamola V, et al. A deep learning based misbehavior classification scheme for intrusion detection in cooperative intelligent transportation systems. *Digit Commun Networks*. 2023;9(5):1113–1122. doi: [10.1016/j.dcan.2022.06.018](https://doi.org/10.1016/j.dcan.2022.06.018)
- [17] Alsubhi K. A secured intrusion detection system for mobile edge computing. *Appl Sci*. 2024;14(4):1432. doi: [10.3390/app14041432](https://doi.org/10.3390/app14041432)
- [18] Javeed D, Gao T, Saeed MS, et al. An intrusion detection system for edge-envisioned smart agriculture in extreme environment. *IEEE Internet Things J*. 2023;11(16):26866–26876. doi: [10.1109/JIOT.2023.3288544](https://doi.org/10.1109/JIOT.2023.3288544)
- [19] Tariq N, Alsirhani A, Humayun M, et al. A fog-edge-enabled intrusion detection system for smart grids. *J Cloud Comp*. 2024;13(1):43. doi: [10.1186/s13677-024-00609-9](https://doi.org/10.1186/s13677-024-00609-9)
- [20] Alzubi OA, Alzubi JA, Qiqieh I, et al. An IoT intrusion detection approach based on salp swarm and artificial neural network. *Int J Network Manag*. 2025;35(1):e2296. doi: [10.1002/nem.2296](https://doi.org/10.1002/nem.2296)
- [21] Alzubi JA, Alzubi OA, Qiqieh I, et al. A blended deep learning intrusion detection framework for consumable edge-centric IoMT industry. *IEEE Trans Consum Electron*. 2024;70(1):2049–2057. doi: [10.1109/TCE.2024.3350231](https://doi.org/10.1109/TCE.2024.3350231)
- [22] TON-IoT dataset. accessed 2025 July. Available from: <https://www.kaggle.com/datasets/dhoo gla/nftoniotv2>

- [23] BOT-IoT dataset. accessed 2025 July. Available from: <https://research.unsw.edu.au/projects/bot-iot-dataset>
- [24] Ullah I, Mahmoud QH. Design and development of RNN anomaly detection model for IoT networks. IEEE Access. 2022;10:62722–62750. doi: 10.1109/ACCESS.2022.3176317
- [25] Donkol AAE-B, Hafez AG, Hussein AI, et al. Optimization of intrusion detection using likely point PSO and enhanced LSTM-RNN hybrid technique in communication networks. IEEE Access. 2023;11:9469–9482. doi: 10.1109/ACCESS.2023.3240109
- [26] Wu Q, Ding K, Huang B. Approach for fault prognosis using recurrent neural network. J Intell Manuf. 2020;31(7):1621–1633. doi: 10.1007/s10845-018-1428-5
- [27] Nejadettehad A, Mahini H, Bahrak B. Short-term demand forecasting for online car-hailing services using recurrent neural networks. Appl Artif Intel. 2020;34(9):674–689. doi: 10.1080/08839514.2020.1771522
- [28] Dong B, Li Y, Gao Y, et al. Multistream regression with asynchronous concept drift detection. In: 2017 IEEE International Conference on Big Data (Big Data); Boston, MA, USA. IEEE; 2017 Dec. p. 596–605.
- [29] Wang L, Petrova K, Yang ML. Trust models in wireless sensor networks for defending against denial-of-service attacks: a literature review. Appl Sci. 2025;15(6):3075. doi: 10.3390/app15063075
- [30] Chen J, Mao F, Lv Z, et al. EdgeFD: an edge-friendly drift-aware fault diagnosis system for industrial IoT. In: 2023 IEEE 23rd International Conference on Communication Technology (ICCT); Wuxi, China. IEEE; 2023 Oct; p. 390–396.
- [31] CSE-CIC-IDS. Dataset. 2018 [cited 2025 Dec 17]. Available from: <https://www.unb.ca/cic/datasets/ids-2018.html>
- [32] Zhukabayeva T, Ahmad Z, Adamova A, et al. An edge-computing-based integrated framework for network traffic analysis and intrusion detection to enhance cyber-physical system security in industrial IoT. Sensors. 2025;25(8):2395. doi: 10.3390/s25082395
- [33] Khan MA. Hcrnnids: hybrid convolutional recurrent neural network-based network intrusion detection system. Processes. 2021;9(5):834. doi: 10.3390/pr9050834